

修士論文

CGLA におけるエネルギー効率の高い ワークロード実行のための AI アプリケーション実装と ソフトウェアスタック改善

安藤 拓翔

2026 年 7 月 30 日

奈良先端科学技術大学院大学
先端科学技術研究科

本論文は、奈良先端科学技術大学院大学先端科学技術研究科における
修士（工学）の学位授与要件の一部として提出したものである。

安藤 拓翔

論文審査委員

Professor Yasuhiko Nakashima	（主指導教員）
Professor Yuichi Hayashi	（審査委員）
Visiting Professor Renyuan Zhang	（審査委員）
Assistant Professor Yirong Kan	（審査委員）
Assistant Professor Pham Hoai Luan	（審査委員）
Assistant Professor Le Vu Trung Duong	（審査委員）

CGLA におけるエネルギー効率の高い ワークロード実行のための AI アプリケーション実装と ソフトウェアスタック改善*

安藤 拓翔

概要

生成 AI、音声認識、画像生成などの AI アプリケーションの利用拡大に伴い、推論時の計算量と消費電力がシステム上の課題になっている。これらの主な計算基盤である GPGPU は高いスループットを提供するが、消費電力が大きい。固定機能アクセラレータはモデル構造や量子化形式の変化に追従しにくい。このため、既存アプリケーションの柔軟性を保ちながら、エネルギー効率の高い実行を可能にするソフトウェアスタックとアクセラレータの連携が必要である。

本論文では、CGLA アーキテクチャを評価する IMAX FPGA プロトタイプ上に、LLM 推論、Whisper ASR、Stable Diffusion の実行経路を実装する。アプリケーションレベルでハードウェア効率を引き出すためのソフトウェアスタック改善も評価する。対象には、既存 OSS アプリケーションの実行フローを保つためのオフロード境界、データ転送方針、ホスト CPU 側制御を含む。本論文の CGLA 評価は低バッチのローカル実行におけるデータ転送、設定、ホスト CPU 側残余時間を含む実行境界を対象とする。評価範囲はモデルロード完了後からアプリケーション出力までであり、アクセラレータ実行、データ転送、ホスト処理を分離する。

*修士論文, 奈良先端科学技術大学院大学 先端科学技術研究科, 2026 年 7 月 30 日。

LLM 推論の評価では、デコード実行が繰り返し発生するデータ転送とホスト CPU 側の処理に強く影響されることを確認した。Q3_K の Qwen3-0.6B, Qwen3-1.7B, Qwen3-8B では、Q-Snap によりプロンプト処理スループットがそれぞれ 1.62 $\times$, 1.36 $\times$, 1.16 $\times$ 向上した。

LLM 推論には llama.cpp を用い、GGML 系の実行基盤を共有する音声認識と画像生成にも同じ実装方針を適用した。whisper.cpp と stable-diffusion.cpp では、量子化行列演算に加えて FP16 行列演算を IMAX 実行対象に含めた。これにより、異なるデータ型と実行時構造を持つアプリケーションで CGLA 実行を評価した。これらの結果から、アプリケーションレベルの効率には、チャンクと転送の粒度を局所メモリ容量に合わせる 것이重要である。

キーワード:

LLM, ソフトウェアスタック, AI アクセラレータ, CGLA, IMAX, エネルギー効率

目次

図目次	vi
表目次	vi
第 1 章 はじめに	1
1.1 研究背景	1
1.2 研究目的	2
1.3 本論文の貢献	3
1.4 本論文の構成	4
第 2 章 関連研究と CGLA/IMAX アーキテクチャ	5
2.1 AI 推論ワークロード	5
2.2 AI 推論の計算基盤	6
2.3 汎用計算デバイスから CGLA への発展	7
2.4 CGLA と IMAX アーキテクチャ	9
第 3 章 GGML 系実行スタックと IMAX 統合	12
3.1 GGML 系アプリケーションスタック	12
3.2 IMAX 統合境界	13
3.3 実行フェーズと測定観点	14

第 4 章	IMAX 上の LLM 実行	18
4.1	対象ワークロードと実行モデル	18
4.2	量子化カーネルマッピング	19
4.3	システムレベルの評価結果	23
4.3.1	プロンプト処理とデコード	25
4.3.2	モデルサイズのスケールリング	27
4.3.3	LMM 容量の設計空間	28
第 5 章	CGLA 向けソフトウェアスタック改善	31
5.1	スタックレベル改善の必要性	31
5.2	Q-Snap	32
5.2.1	チャンクサイズ導出	33
5.2.2	定量結果	34
5.3	プロファイリング基盤	36
5.4	スタック改善から得られる設計規則	36
第 6 章	IMAX 上の音声認識と画像生成	38
6.1	GGML 系アプリケーションへの展開	38
6.2	Whisper ASR	39
6.2.1	FP16 カーネル設計と実装技法	39
6.2.2	エネルギー効率とプラットフォーム比較	41
6.3	Stable Diffusion	45
6.3.1	実験設定と定量結果	45
6.4	ワークロード横断の考察	47
6.5	評価範囲と限界	47
第 7 章	結論	49

謝辞	51
参考文献	52
付録 A 発表リスト	57
付録 B 受賞	59

図目次

2.1	メニーコア／GPGPU と CGLA の構造比較。	8
2.2	IMAX アーキテクチャとレーン構造の概要。	10
3.1	三層ソフトウェアスタック。	14
4.1	LLM 推論のタスク分割。	21
4.2	Q6_K と Q3_K ドット積データフロー。	22
4.3	Qwen3 構成のエンドツーエンドレイテンシ。	25
4.4	IMAX 上のプロンプト処理とデコードのフェーズ内訳。	26
4.5	Qwen3-1.7B Q8_0 のレイテンシと PDP。	27
4.6	Qwen3-0.6B Q3_K_S の PDP と EDP。	28
4.7	Qwen3 ワークロードの LMM サイズ PDP 掃引。	29
5.1	Q-Snap, 既定ヒューリスティック, 固定チャンクの比較。	32
5.2	Q-Snap のスループット改善。	35
5.3	Q-Snap のチャンクサイズ感度。	36
6.1	IMAX 上の FP16 ドット積カーネルのデータフロー。	40
6.2	Whisper ASR 処理フロー。	41
6.3	Whisper ASR の PDP とバースト長トレードオフ。	43
6.4	Whisper ASR における LMM サイズのトレードオフ。	44
6.5	SD-Turbo の IMAX と RTX 4090 の比較。	46

表目次

4.1	Qwen3 評価プラットフォームと IMAX ASIC 投影の物理仕様。	24
6.1	LMM サイズ別の Whisper カーネル累積対応率。	44

第 1 章

はじめに

1.1 研究背景

AI モデルの大規模化に伴い、システム全体での消費電力が課題となっている。大規模言語モデル (LLM)、音声認識、画像生成の推論では、行列演算、モデルパラメータの移動、ホスト CPU 側制御が繰り返し発生する。そのため、消費電力は瞬間的なピークではなく持続的な問題になる。

GPU (Graphics Processing Unit) は LLM 推論等の AI ワークロードで高いスループットと成熟したソフトウェアスタックを持つため広く用いられるが、消費電力は大きい。固定機能に特化した ASIC (Application-Specific Integrated Circuit) は低消費電力を実現し得るが、モデル構造や量子化形式の変化への追従が難しい。そこで、先行研究では、処理要素 (PE) と Local Memory Module (LMM) を一次元に配置する CPU-Grounded Linear Array (CGLA) アクセラレータである IMAX を提案した [1, 2]。本論文で CGLA を評価する IMAX3 は、その FPGA プロトタイプである。本論文では、この CGLA 上で既存のオープンソースソフトウェア (OSS) アプリケーションの実行経路を実装・評価し、低バッチのローカル実行における支配的カーネル、転送粒度、ホスト CPU 側制御を同じ境界で扱う。

本論文は、llama.cpp [3], whisper.cpp [4], stable-diffusion.cpp [5] の三つの OSS スタックを評価する。これらは、それぞれ LLM 推論、音声認識、画像生成を担う。これらは共通のバックエンド GGML を持つため、本論文はその GGML 上の支配的な計算カーネルを IMAX バックエンドへオフロードする。

1.2 研究目的

本論文では、LLM 推論、Whisper による自動音声認識、Stable Diffusion による画像生成の CGLA 実行経路を実装する。評価には、カーネル実行だけでなく、アクセラレータ設定、データ転送、ホスト CPU フォールバック、残余ホスト CPU 実行を含める。実装範囲には、CGLA 実行に必要なオフロード境界、転送方針、フォールバック処理、プロファイリングでの可視性を含める。

評価では次の三点を問う。

1. 三つの GGML 系アプリケーションをフロントエンドの置換なしで実行するには、アプリケーション層、オフロード層、CGLA 層のどの構成要素が必要か。
2. 評価した Qwen3 経路では、どの測定フェーズがプロンプト処理とデコードを制約し、Q-Snap はプロンプト処理スループットをどのように変えるか。
3. Whisper と Stable Diffusion が同じ GGML オフロード境界を用いる場合、どの量子化演算と FP16 演算が IMAX 上で実行されるか。その条件下で得られる正しさ、レイテンシ、PDP は何か。

評価は、チャンク分割、Direct Memory Access (DMA) 転送、ホスト CPU 側フォールバックを含むエンドツーエンド実行に基づく。CGLA 経路は、これらのコストを含めた上で正味の効果を示す必要がある。

1.3 本論文の貢献

本論文の主な貢献は以下の通りである。

- llama.cpp, whisper.cpp, stable-diffusion.cpp に、モデル読み込み、テンソル管理、コマンドラインインターフェース、アプリケーション制御を保った CGLA 実行経路を実装した。
- llama.cpp に量子化ドット積のオフロード経路とフェーズ別プロファイリングを実装し、評価した Qwen3 条件では LOAD とホスト処理がデコードの主要コストとなることを確認した。
- Q-Snap をスタックレベル方針として実装し、評価対象でプロンプト処理のスループットを $1.62\times$ 改善した。
- Whisper 向け FP16 オフロードを実装した。Whisper-tiny.en と LibriSpeech test-clean の 21 発話を用いた評価では、FP16 と Q8_0 の平均正規化 CPU-IMAX 文字起こし差は 0.00% と 0.13% であった。
- Stable Diffusion では Q3_K, Q8_0, FP16 のドット積に同じバックエンドを用いた。評価した FP16 形状は guard 領域を破壊せず、CPU 参照値と一致した。

すべての評価対象で、アプリケーションとオフロード層を統合した後のアプリケーションレベル性能を同じ境界で測定する。この境界には、アクセラレータ設定と転送、ホストフォールバック、残余ホスト実行を含める。

1.4 本論文の構成

第 2 章では、アーキテクチャと関連研究の背景を述べる。第 3 章では、ソフトウェアスタック観点と評価章で用いる測定境界を定義する。第 4 章では、IMAX 上の LLM 実行を分析する。第 5 章では、同じ llama.cpp 経路におけるソフトウェアスタック改善として Q-Snap を評価する。第 6 章では、Whisper ASR と Stable Diffusion を追加のアプリケーション経路として評価する。最後に、第 7 章で本論文をまとめる。

第 2 章

関連研究と CGLA/IMAX アーキテクチャ

本章では、後続の実装章に必要なワークロードと計算基盤の背景を整理する。その後、CGLA と IMAX の構成を説明し、IMAX を用いた LLM 先行研究に対する本論文の位置付けを示す。

2.1 AI 推論ワークロード

現代の LLM 推論では、行列積、重み移動、実行時制御が支配的である。Transformer アーキテクチャ [6] は、マルチヘッド attention と大きな feed-forward ブロックを中心に置く。その後のモデル規模拡大により、計算量とメモリ需要が増加した [7]。プロンプト処理は大きな行列演算を含む。デコードは小さな演算を繰り返すため、転送オーバーヘッドと固定コストの影響を強く受ける。このフェーズ差は、IMAX では転送粒度とホスト制御が演算カーネル高速化の効果を左右するため重要である。

llama.cpp スタックは、量子化対応、バックエンド選択、テンソル管理、アプリ

ケーションレベル推論を統合する [3]。GPTQ [8], LLM.int8() [9], AWQ [10] などの post-training 量子化は、メモリフットプリントを抑えた LLM 実行の基盤を作った [11]。GGML は複数の量子化形式をサポートしており、同一スタックから異なる精度のモデルを読み込んで実行できる。

本論文では、LLM 推論向けに構築した CGLA 実行経路が、異なるデータ型と実行時構造を持つアプリケーションにも適用できるかを確認するため、Whisper ASR と Stable Diffusion も評価する。Whisper [12] は音声認識向けの transformer encoder-decoder モデルであり、whisper.cpp [4] はその C/C++ 推論経路を提供する。Stable Diffusion は Latent Diffusion Model [13] に基づき、stable-diffusion.cpp [5] は本論文で評価する C/C++ 推論経路を提供する。どちらも llama.cpp と同じ GGML バックエンドを用いるため、CGLA 統合ではテンソルおよびバックエンドインターフェースを再利用できる。これらは LLM 推論とは異なるが、ホスト CPU 側制御、局所メモリ容量、DMA 挙動、カーネル対応範囲という統合上の課題を共有する。

2.2 AI 推論の計算基盤

CPU は汎用的なプログラム可能性を持つが、密なテンソル演算に対するエネルギー効率に限定的である。GPU は高スループットと成熟したソフトウェアエコシステムを持つ一方で、消費電力が大きい。広幅 SIMD (Single Instruction, Multiple Data) 実行ユニット、大規模レジスタファイル、スループット重視の高帯域メモリ階層が GPU の高電力消費の主な要因となっている。HBM を搭載する GPU はメモリスループットを高められるが、本論文が扱う低バッチ推論では、起動、転送、ホスト側制御の固定コストもエンドツーエンド時間に残る。AI 推論では行列積と重み移動が支配的であり、演算精度を下げてでも認識精度の劣化が小さいことが知られている [11]。この性質は専用ハードウェアによる省電力化の余地を生

むが、モデル構造と量子化形式が変化し続けるため、専用化と汎用性のどちらを取るかが計算基盤選択の中心的な問題になる。

2.3 汎用計算デバイスから CGLA への発展

本節では、汎用プロセッサと GPU、ドメイン固有アーキテクチャ、粗粒度再構成可能アーキテクチャ（Coarse-Grained Reconfigurable Architecture: CGRA）を順に概観する。その後、CGLA を必要とする制約を整理する。

汎用プロセッサはアウトオブオーダー（out-of-order, OoO）実行や投機実行に多くの面積と電力を割くが、ループアンローリングが可能で複雑な分岐を持たない密なテンソル演算は、これらの機構の恩恵をほとんど受けない。GPU は投機機構を持たず、SIMT（Single Instruction, Multiple Threads）方式で多数の SIMD 演算器を動作させる。Warp ストールを隠すには、広帯域メモリ、コアレッシング、大きなレジスタファイルとキャッシュが必要である。ハードウェアがデータの流れを制御するためプログラミングは容易になる一方、組み込み機器への実装では面積と電力のオーバーヘッドが大きい。メニーコア／汎用 GPU（GPGPU）と CGLA の構造の違いを図 2.1 に示す。メニーコア／GPGPU では、多数の PE が共有メモリ階層を介してデータを受け取る。これに対して CGLA は、PE と LMM を一次元に配置し、主記憶との PREFETCH／DRAIN と隣接ユニット間の一方向転送によってデータ移動を明示的に制御する。

ドメイン固有アーキテクチャ（Domain-Specific Architecture, DSA）は、低精度の固定小数積和、深い演算パイプライン、データ再利用による主記憶アクセス削減により、高い電力あたり性能を達成できる [14, 15]。この設計は、推論では演算精度を落としても認識精度の低下が小さいこと [11]、および主記憶アクセスのエネルギーコストが大きいこと [16] に支えられている。固定機能 DSA は、対応する演算を狭い対象に限定する。このような固定機能設計は、対応範囲外の変更にハー

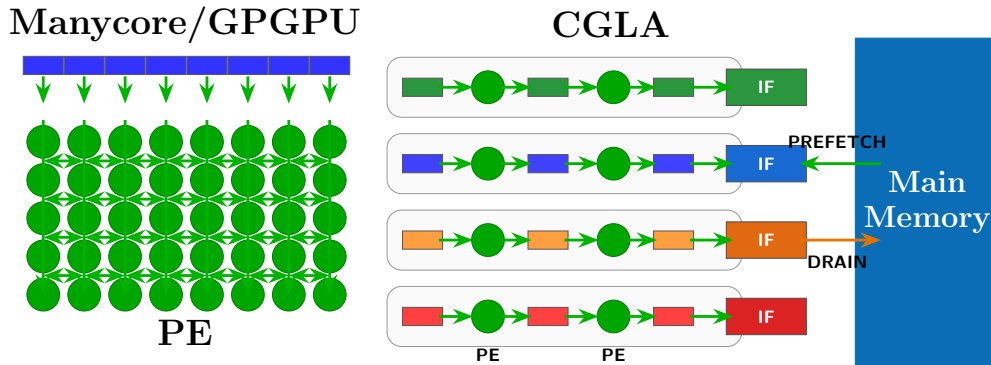


図 2.1 メニーコア／ GPGPU と CGLA の構造比較。共有メモリ階層から多数の PE へデータを供給する構成と， PE/LMM 列へデータを一方向に流す構成の違いを示す。

ドウェア再設計なしで追従できない。半導体コストが上昇するなかで， 応用ごとに ASIC を設計しなおすことは現実的でない。したがって， GPU より省電力で， か
つプログラマビリティを残したアクセラレータが求められる。

CGRA は， 隣接接続を含むトポロジ依存の配線網により粗粒度演算ブロックを空間的に接続する [2, 17]。 TPU [18] が採用するシストリックアレイも規則的な演算に空間的なデータ移動を用いるが， 一般の CGRA と同一ではない。 TPU に代表されるシストリックアレイは規則的な密行列演算を高い稼働率で処理するが， 実行形状， 量子化形式， ホスト側のアプリケーション制御が変わる場合には， ソフトウェア境界の扱いが性能を左右する。 ステンシル演算やベクトル演算を対象とする EMAX 系は， 演算ユニット内に LMM とアドレス計算回路を持つ。 主記憶間のバースト転送と演算をオーバラップし， 高い演算器稼働率を保つ [19, 20]。 ただし二次元 CGRA は膨大な配線を要して面積効率を下げ， LSI 実装を難しくする。 また不連続アクセスのために同一データを複数の LMM へブロードキャストする方式は， メモリ面積のコストが大きい [20]。

IMAX は， CGLA アーキテクチャを PE と LMM の一次元交互配置とリング接続で実現し， 隣接ユニット間でデータを一方向に転送する [1]。 二次元の配置配線を

必要としないため、配線負荷を抑えられる。各ユニットは、互いに重複しないメモリ領域を保持する。そのため、キャッシュラインの相互追い出しと、それを隠す大規模なマルチスレッディング機構が不要になる。未使用 LMM はクロックゲーティングでき、省電力につながる。こうして CGLA は、変化するモデル構造や量子化形式へのプログラマビリティを保ちながら、GPU より低い電力を目指す。本論文では、局所メモリに収まる計算単位、明示的な DMA、ホストフォールバックを含む実行境界を制御できる点から CGLA を用いる。この性質により、カーネル、転送、ホスト処理のどれがエンドツーエンド挙動を支配しているかを同じアプリケーション経路上で分離できる。本論文は CGLA を対象とし、IMAX アクセラレータの FPGA プロトタイプである IMAX3 を用いて AI ワークロードを実行する。先行研究は IMAX を用いた CGLA 上での LLM 実行とカーネル対応範囲を示している [21, 22]。本論文では、評価対象を音声認識と画像生成へ拡張する。llama.cpp のソフトウェアスタックには Q-Snap を実装する。これらを同一のエンドツーエンド測定境界で評価する。

2.4 CGLA と IMAX アーキテクチャ

CGLA の正式名称は CPU-Grounded Linear Array であり、プログラム可能なアクセラレータアーキテクチャである。IMAX は、節 2.3 で導入した PE/LMM の一次元リングにより CGLA を実現する [1, 2]。図 2.2 は、IMAX のシステム構成とレーン内の PE/LMM 配置を示す。ホストから供給されたデータは各レーンの LMM に格納され、隣接する PE 間を流れるパイプラインとして処理される。

CGLA は、典型的な二次元 CGRA のような RISC 型ユニットではなく、CISC 型の処理要素を採用している。CISC 型 PE は 1 つのハードウェア操作により多くの処理を詰め込めるため、操作列が短くなり依存チェーンが縮まる。カーネルは二次元の配置配線を必要としない。各カーネルは PE と LMM のペア列へ順番に対

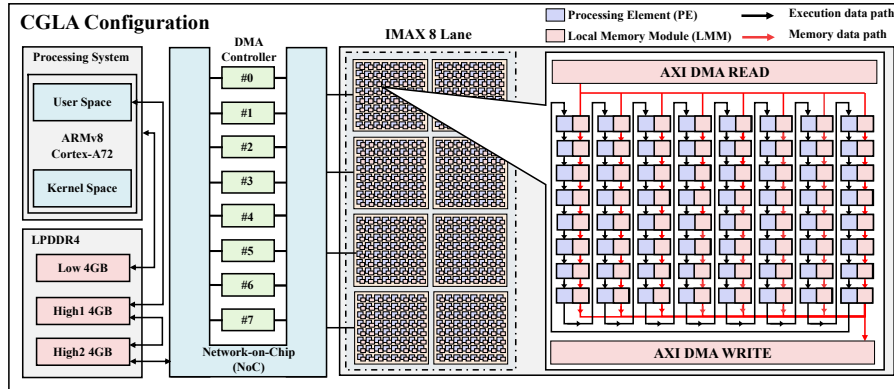


図 2.2 IMAX アーキテクチャとレーン構造の概要。ホスト，DMA，複数レーン，および各レーン内で交互に配置された PE と LMM の接続関係を示す。

応付けられる。対応付け位置は互いに独立しているため，コンパイルに探索が不要である。ハードウェアにマップする前に同じ操作列を CPU 上で実行できるため，アルゴリズムレベルの事前検証が容易である。

各カーネル呼び出しは固定のフェーズ列に従う。CONF は PE アレイと LMM を設定する。LOAD は DMA 経由でホストメモリから入力データを転送する。EXEC はパイプライン演算を実行する。DRAIN は結果をホストへ戻す。理想的には，あるイテレーションの DRAIN と現在の EXEC バーストが重複する。次の LOAD プリフェッチも同時に進み，パイプラインを稼働状態に保つ [1]。実効スループットはこの重複が持続できるかどうか依存し，チャンクサイズ，LMM 容量，DMA 粒度が決め手となる。

IMAX は PE と LMM を一次元に配置した CGLA アクセラレータであり，本論文で用いる IMAX3 はその FPGA プロトタイプである [1]。CGLA チップは AXI スレーブインターフェースでカスケード接続でき，専用外部メモリバスなしに複数チップにまたがるパイプラインを構築できる。IMAX シリーズは LAPP から EMAX 系を経て IMAX へと発展しており，各世代で LMM 容量，DMA スループット，面積効率，マルチチップ対応が改善されてきた [1]。

本論文の主な FPGA 基準である IMAX3 は、AMD Versal VPK180 FPGA とオンチップ ARM Cortex-A72 ホストを用い、8 レーン、各レーン 64 PEs、各レーン 512 KiB LMM を持つ。ASIC 投影では、Synopsys Design Compiler [23] による 28 nm の静的タイミングおよび合成推定を参照し、IMAX active の FPGA 実測時間を 145 MHz から 840 MHz の条件へ換算する。

GPU と比べて、IMAX はデータ移動をより明示的にソフトウェアへ露出する。この制御はメモリ階層のオーバーヘッドを下げ得るが、同時に局所メモリ容量、DMA 粒度、ホスト側制御を主要な設計問題にする。したがって、同じ演算カーネルでも、チャンク分割と転送方針によりエンドツーエンド挙動が変わる。上谷 [21] は CGLA 上での LLM 推論が実現可能であること、および支配的なドット積カーネルをアクセラレータにマップできることを示した。衛藤 [22] はこれを拡張し、LLaMA 系モデルについて異なる量子化形式下でのカーネル対応範囲を評価した。これらの先行研究は、LLM 実行とカーネル対応範囲を示した。しかし、複数の GGML 系アプリケーションに対する同一フェーズでの比較は含まれていない。比較対象は、オフロード境界、チャンク形成、DMA、ホスト CPU 側フォールバックである。本論文は、共通 GGML バックエンドを Whisper ASR と Stable Diffusion へ拡張する。Q-Snap はソフトウェアスタック改善として実装する。これらを同一のエンドツーエンド測定境界で評価する。

第 3 章

GGML 系実行スタックと IMAX 統合

本章では，各アプリケーション評価に共通するソフトウェア境界と測定境界を定義する。アプリケーション固有の実装と評価結果は，第 4 章と第 6 章で扱う。

3.1 GGML 系アプリケーションスタック

本論文の共通方針は，既存 OSS アプリケーションスタックを再利用し，IMAX 統合に必要な部分だけを変更することである。モデル読み込み，テンソル管理，CLI 挙動，アプリケーションロジックの大部分は元のアプリケーションに残す。計算量の大きいカーネルだけを選択的に CGLA バックエンドへオフロードする。この方針により，評価を実際のソフトウェア利用に近づけられる。同時に，IMAX 統合の主要な問題も見える。エンドツーエンド挙動は演算カーネルだけでなく，量子化形式，チャンク分割方針，DMA 制御，ホスト側実行時挙動，フォールバック処理に依存する。

本論文が対象とする実行スタックは，三つのアプリケーションが用いる GGML

系 C/C++ スタックである。対象は llama.cpp, whisper.cpp, stable-diffusion.cpp である。これらのアプリケーションは、従来のフロントエンド実行フローを保つ。演算形状、データ型、転送サイズに応じて、ホスト実行、IMAX 実行、混在実行を選ぶ。この境界では、既存アプリケーションの実行経路と統合後の実行コストを評価する。

3.2 IMAX 統合境界

実行スタックは三層で捉える。アプリケーション層は、本論文で扱う三つのアプリケーションからなる。オフロード層はバックエンドディスパッチ、チャンク形成、転送スケジューリング、フォールバック処理、プロファイリングを担う。ハードウェア層は IMAX 上でカーネルを実行し、局所メモリモジュールを管理する。図 3.1 は、三層の責務と、GGML 系アプリケーションから IMAX バックエンドまでの接続関係を示す。アプリケーション層は既存の推論フローを保持する。オフロード層は、演算の形状とデータ型に基づいて実行先を選択する。ハードウェア層は、選択されたカーネルと転送を実行する。

多くのボトルネックは、これらの層の境界で発生する。たとえば、アプリケーションレベルの量子化選択は、オフロード層から見えるデータ配置を変える。オフロード層は、ハードウェアが転送コストと制御コストを何回支払うかを決める。単独カーネルの実行時間が短くても、転送と制御を含むエンドツーエンド時間が短くなるとは限らない。

本論文の実装成果はオフロード層に集中する。LLM カーネルの IMAX バックエンド経路、Whisper ASR の FP16 経路、Stable Diffusion バックエンド挿入、Q-Snap チャンク選択、フェーズ別プロファイリング用フックが含まれる。以降の章では、これらの実装をアプリケーション実行経路内で評価する。本スタックは線形データフローモデルに合うカーネルを選択的にオフロードし、残りはホストに残

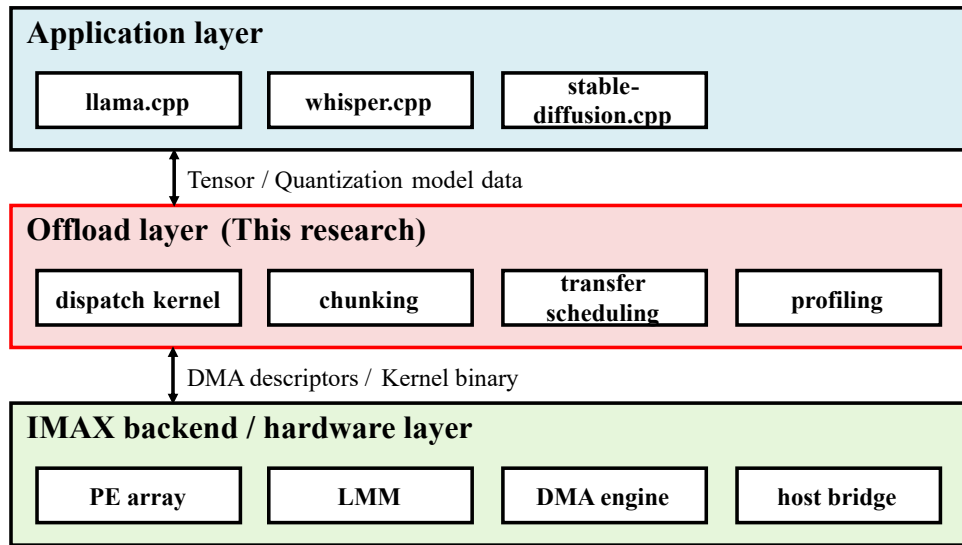


図 3.1 llama.cpp, whisper.cpp, stable-diffusion.cpp を IMAX バックエンドに接続する三層ソフトウェアスタック。既存アプリケーションを保持するアプリケーション層，実行先と転送を制御するオフロード層，カーネルを実行するハードウェア層を示す。

す。対象カーネルが選ばれると，オフロード層は IMAX の局所メモリモジュールに収まるチャンクを形成し，対応する転送を発行する。チャンク境界はバーストサイズ，DMA 回数，量子化アラインメント，設定オーバーヘッドを決めるため，チャンク分割を実行時方針として扱う。未対応演算では CPU フォールバックが必要であり，この処理は同期と制御遷移を追加する。この同期と制御遷移は，主なカーネルがオフロードされていてもエンドツーエンド挙動を悪化させ得る。ホストコストと転送コストは連動しており，一方を下げるともう一方が支配的ボトルネックとして現れる場合がある。

3.3 実行フェーズと測定観点

IMAX の実行時間は，二つの測定境界で扱う。一つは，アプリケーション全体を測るエンドツーエンド境界である。もう一つは，オフロードされたカーネル内部

を測るフェーズ境界である。カーネル内部では、ホストによる設定、DMA 転送、PE アレイ上の演算を個別に測定する。CONF は、ホスト CPU が Programmed I/O (PIO) を介して IMAX のマッピング命令と実行制御情報を書き込む時間である。CONF は、入力データの LOAD とは別の制御転送である。短いチャンクで繰り返すと、演算時間とは独立にエンドツーエンド時間へ残る。REGV は、カーネルが使用する PE の内部レジスタを PIO で初期化する時間であり、使用する PE 数と設定値の量に依存する。RANGE は、LMM 上で各入力、出力、作業領域が使用するアドレス範囲を設定する時間である。LOAD は、ホスト主記憶上の連続 DMA バッファから入力データを LMM へ転送する時間である。複数の入力配列が分散している場合は、ホスト側で一つの連続領域に集約してからバースト転送し、DMA 発行回数を抑える。EXEC は、設定済みの PE アレイが LMM のデータを読み出し、SIMD 命令を含むパイプラインとして対象カーネルを実行する時間である。DRAIN は、演算結果を LMM からホスト主記憶上の DMA バッファへ戻す時間である。

IMAX の LMM はダブルバッファ構成を持つ。一方のバッファに対する EXEC と、もう一方のバッファに対する次の LOAD または直前結果の DRAIN を重ねられる。したがって、フェーズ時間は単純な逐次処理の内訳ではなく、転送と演算を重複させた状態で残る待ち時間を表す。LOAD の割合が大きい場合は入力データ量、DMA バースト粒度、ホスト・アクセラレータ間帯域が律速要因となる。EXEC の割合が大きい場合は PE 上の演算パイプラインが主要時間を占める。DRAIN の割合が大きい場合は出力配置、転送回数、書き戻し経路が律速要因となる。CONF、REGV、RANGE の割合が大きい場合は、カーネル実行そのものではなく、短いチャンクに対して設定を繰り返す制御粒度に改善余地がある。この分類により、演算カーネルの変更、チャンク分割、ホスト CPU 側制御改善のどれが必要かを区別する。

ソフトウェアスタックには、計算時間、転送、ホストオーバヘッドを分離する観測性が必要である。本論文では、LOAD, DRAIN, EXEC, CONF, REGV, RANGE のフェーズ別時間と、オフロード外のホスト実行時間を用いる。同じ測定境界を、各比較内の全プラットフォームと LLM, Whisper ASR, Stable Diffusion に適用する。タイマはモデルロード完了後に開始し、CPU による後処理を経たアプリケーション出力の生成時点で停止する。測定には、アクセラレータ設定と転送、ホストフォールバック、残余ホスト実行を含める。モデルロードは、ストレージやページキャッシュ状態の影響を避けるため除外する。CPU とメモリ構成がプラットフォームごとに異なるため、本論文の比較単位はシステムレベルである。

主な指標は、エンドツーエンドレイテンシ、フェーズ別実行内訳、推定実行エネルギー (PDP), EDP, および利用可能な場合の正しさ、カーネル対応率、LMM 容量感度である。単一の消費電力を用いるプラットフォームでは、PDP と EDP を消費電力およびエンドツーエンドレイテンシから次式で定義する。

$$\text{PDP} = \text{消費電力} \times \text{エンドツーエンドレイテンシ}. \quad (3.1)$$

$$\text{EDP} = \text{消費電力} \times \text{エンドツーエンドレイテンシ}^2. \quad (3.2)$$

IMAX ASIC の投影では、PDP を次式で計算し、IMAX active 時間とホスト CPU 側残余時間を分けて扱う。

$$\text{PDP}_{\text{IMAX}} = P_{\text{IMAX}}T_{\text{IMAX}} + P_{\text{ARM}}T_{\text{ARM}}, \quad (3.3)$$

ここで、 P_{IMAX} と P_{ARM} は消費電力、 T_{IMAX} と T_{ARM} はそれぞれ IMAX active 時間およびホスト CPU 側残余時間を表す。IMAX の EDP は、この PDP に投影後のエンドツーエンドレイテンシを掛けた値である。本論文は、既存推論アプリケーションにおけるカーネルオフロードのシステム全体への影響を評価する。計算結果を保存するソフトウェアスタック変更では、アプリケーション品質を成立

条件として扱う [24]。したがって、共通のエンドツーエンド境界におけるレイテンシとエネルギーのトレードオフを表す PDP と EDP を主要指標とする。これらは、フェーズ内訳、正しさ、カーネル対応率、LMM 容量感度、および合成に基づく IMAX 投影条件と併せて解釈する。Whisper 経路では単語誤り率 (word error rate: WER) と文字誤り率 (character error rate: CER) を用い、Stable Diffusion カーネルでは CPU 参照値との一致を確認する。LLM 比較ではモデルと量子化形式を固定し、量子化形式間のモデル品質の優劣を主張しない。

第 4 章

IMAX 上の LLM 実行

本章では、本論文の中核となる実装と評価を示す。まず llama.cpp の実行経路を示し、そこで観測したボトルネックを基に、第 5 章で評価するソフトウェアスタック改善へ接続する。

4.1 対象ワークロードと実行モデル

本章では、llama.cpp を用いた IMAX 上の LLM アプリケーション経路を扱う。基本方針は、従来の実行構造を保ち、支配的な計算カーネルのみを CGLA バックエンドへオフロードすることである。この方針により、広く用いられている推論ソフトウェアの中で IMAX が動作できるかを評価する。

システムレベル挙動はカーネルマイクロベンチマークだけでは推定できない。LLM 推論ではプロンプト処理とデコードでボトルネックが異なり、チャンク分割、DMA、ホスト制御、量子化形式の相互作用がエンドツーエンド挙動を大きく左右する。本章の主評価では、複数のモデル規模を同じ llama.cpp スタックで実行できる Qwen3 系列 (0.6B, 1.7B, 8B) を用いる。Qwen3 結果は、エンドツーエンドレイテンシ、エネルギー指標、LMM サイズ分析に用いる。Llama3-8B Q3_K_S 結果

は、実ソフトウェアスタックにおける対応範囲とフォールバックの注記としてののみ用いる。

4.2 量子化カーネルマッピング

主要な実装対象は、GGML 実行経路で支配的な量子化ドット積カーネルである。本論文では、Q8_0, Q6_K, Q3_K の量子化形式に対するドット積カーネルを IMAX の一次元データフローへマップする。高位の制御、テンソル管理、KV-cache ロジックはホスト CPU に残し、行列積から呼び出される量子化ドット積をオフロードする。

Q8_0. Q8_0 は、32 要素の重みを符号付き 8 bit 値とブロックスケールで表現する。カーネルは量子化重みと 8 bit 入力を LMM から読み出し、SML8 命令で二つの 8 bit 乗算を並列に処理する。部分和は 24 bit 整数として PE 列を流れ、AD24 命令で累積した後にブロックスケールを適用して FP32 出力へ変換する。このデータフローは四並列で動作し、二回の実行で 32 要素を処理する。カーネルは 46 PEs を使用する。Q8_0 の整数積和とスケール適用の流れを、より低 bit の Q6_K と Q3_K に共通する後段パイプラインの基礎とする。

Q6_K. Q6_K は、256 要素の重みを上位 2 bit と下位 4 bit に分割して格納し、8 bit スケールを持つ。前段の CVT86 命令は、分離された 2 bit と 4 bit の重みおよびスケールを読み出し、積和演算に用いる中間表現へ一サイクルで展開する。展開後は、SML16 命令で 8 bit 入力との積和を行う。四つのデータフローを並列に動作させ、16 回の反復で 256 要素を処理する。このカーネルは 64 PEs を使用するため、REGV を含む設定コストとレーン内の PE 使用率が他形式より大きい。

Q3_K. Q3_K は、256 要素の重みを 1 bit の上位部と 2 bit の下位部に分け、6 bit スケールと組み合わせる。前段の CVT53 命令は、分割された重みを統合した 3 bit 表現へ再構成し、スケールを後段の SIMD 演算で扱える形式へ変換する。そ

の後は、Q6_K と同様に SML16 を用いた整数積和と FP32 への変換を行う。四並列のデータフローを 16 回反復し、256 要素を処理する。Q3_K カーネルは 51 PEs を使用し、低 bit 重みの展開処理と共通積和パイプラインを一つの PE 列内で連続して実行する。

カーネルが LMM から読み出す Q8_0, Q3_K, Q6_K の格納コストは、それぞれ 8.50, 3.44, 6.56 bit/weight である。これらの値はブロックスケールを含み、FP16 の 16.00 bit/weight より小さい。一方、Q3_K と Q6_K は重み本体を分割して格納するため、前段に CVT53 または CVT86 による再構成が必要になる。したがって、量子化は LMM 使用量と転送量を減らす一方で、形式固有の展開処理を PE 列へ追加する。

図 4.1 は、ホストと IMAX バックエンドのタスク分割境界を示す。図 4.2 は、IMAX にマップした Q6_K と Q3_K ドット積カーネルの 1 列分の概要データフローを比較する。両カーネルとも、形式依存の展開処理の後に共通の乗算加算パイプラインを再利用し、展開段と演算段は形式ごとのカスタム命令で実装する。

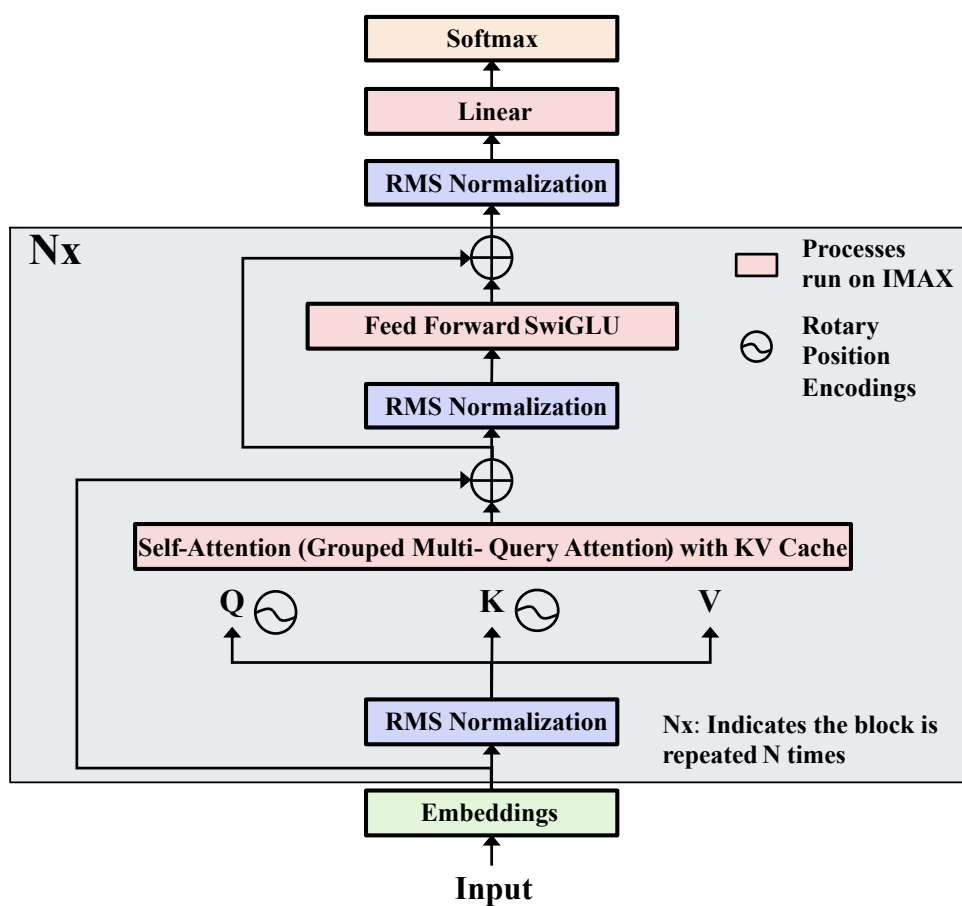


図 4.1 llama.cpp と IMAX バックエンドのタスク分割。ホストはモデル管理，実行制御，非オフロード演算を担当する。IMAX は，GGML の行列積から呼び出される量子化ドット積カーネルを実行する。

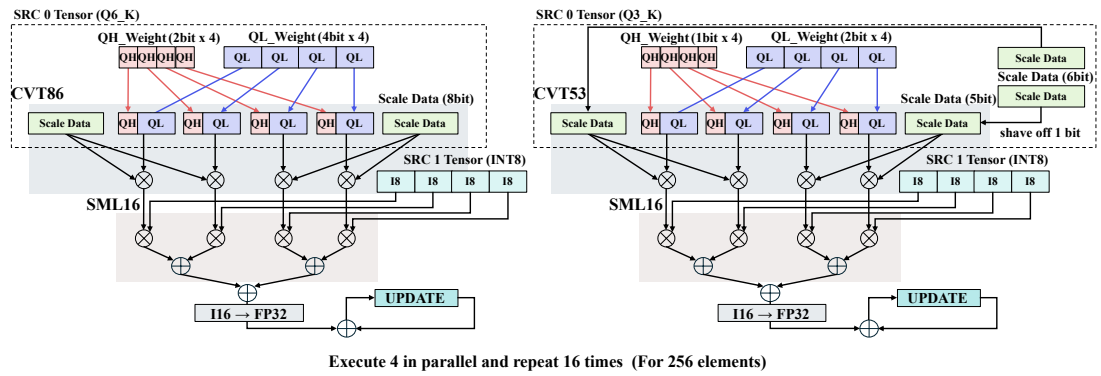


図 4.2 IMAX にマップした Q6_K と Q3_K ドット積カーネルの一系列分のデータフロー。各形式に固有の重み展開とスケール処理を前段で行う。後段では、SIMD 整数積和と FP32 への変換を共通して実行する。

この図はオフロード境界も明示する。IMAX は支配的量子化ドット積カーネルを実行し、ホストはテンソル管理、KV-cache 処理、高位の実行制御を保持する。したがって、IMAX 上の LLM 推論の品質は、マップしたカーネルだけでなく、周辺のオフロード経路の挙動にも依存する。

評価した Q3_K_S 経路では、実行の約 10% がホスト側の Q6_K 計算に依存していた。IMAX 側の Q6_K カーネルを追加すると、32 token の Llama3-8B Q3_K_S レイテンシは 34.2s から 30.4s に低下した。

4.3 システムレベルの評価結果

本節では、量子化カーネルのオフロード後に支配的となる実行フェーズを評価する。また、モデル規模と LMM 容量がレイテンシおよび投影エネルギーに与える影響を評価する。主な LLM 評価では、Qwen3-0.6B, Qwen3-1.7B, Qwen3-8B を Q8_0 と Q3_K_S 構成で評価した。Qwen3 の各図では、横軸の各ラベルが prompt token 数と generated token 数の組を表す。既定のハードウェア条件は VPK180 ベースの IMAX3 プロトタイプであり、8 レーン、各レーン 64 PEs、各レーン 512 KiB LMM、145 MHz の FPGA 動作、ホスト 2 スレッドを用いる。ASIC 投影では、28 nm の Design Compiler 推定 [23] を参照し、IMAX active の FPGA 実測時間を 145 MHz から 840 MHz の条件へ換算する。ホスト CPU 側実行時間は換算しない。PDP と EDP には、この実行時間と合成に基づく IMAX 推定およびプラットフォームの公称電力推定を用いる。prompt token 数と生成 token 数は各図のラベルに示し、各比較では同じホストスレッド数と IMAX プロトタイプ構成を用いた。

表 4.1 に、Qwen3 評価プラットフォームと IMAX ASIC 投影の物理仕様をまとめる。

表 4.1 Qwen3 評価プラットフォーム, IMAX ASIC 投影の物理仕様, および電力モデル。
ASIC 投影には 28 nm の Design Compiler 推定を用いる。

Device	Host CPU	Compute units ^a	Process (node)	Freq. (MHz)	Memory	Power model (W)
IMAX3 (VPK180)	Cortex-A72	64	7	145	8 GB+4 GB ^b	–
IMAX (28nm, 1 lane)	–	64	28	840	–	0.647 ^d
RTX 4090	Xeon W-2455X	16384	4N	2520	24 GB	650 ^d
GTX 1080 Ti [25]	Xeon W-2455X	3584	16	1582	11 GB GDDR5X	250 ^c
Jetson AGX Orin	Cortex-A78	1792	8	930	32 GB	15 ^d

^aIMAX3 と IMAX ではレーン当たりの PE 数を示す。RTX 4090, GTX 1080 Ti, Jetson AGX Orin では CUDA core 数を示す。 ^b8 GB DDR4 は OS バッファ, 4 GB DDR4 は DMA バッファ。 ^cGTX 1080 Ti の値は公称ボード電力であり, アーカイブしたレイテンシ比較の追跡用としてのみ示す。 ^d再計算した PDP/EDP モデルで用いる値であり, IMAX 投影では残余 ARM 実行に 0.6485 W をさらに用いる。

図 4.3 と 図 4.5 のアーカイブした GTX 1080 Ti レイテンシ系列は, 表 4.1 に示す Xeon W5-2455X ホストと, 各図に示すモデル, 量子化形式, トークン構成を用いる。250 W の公称ボード電力は追跡用に示すが, 現行の再計算 PDP および EDP 図は IMAX 投影, RTX 4090, Jetson AGX Orin のみを電力モデルに含めるため, GTX 1080 Ti を除外する。

評価した Qwen3 構成のデコードは, 演算スループットよりもデータ転送とホスト側オーバーヘッドに強く制約される。LOAD, CONF, DRAIN, ホスト残余時間は, 演算器の周波数やレーン数とは別に残る。そのため, エンドツーエンドレイテンシは単純比例で短縮しない。

図 4.3 は, 小さい Qwen3-0.6B Q3_K_S と大きい Qwen3-8B Q8_0 のエンドツーエンドレイテンシ比較を示す。8B 規模では, 重み転送量の増加により GPU プラットフォームとの絶対レイテンシ差が大きくなる。

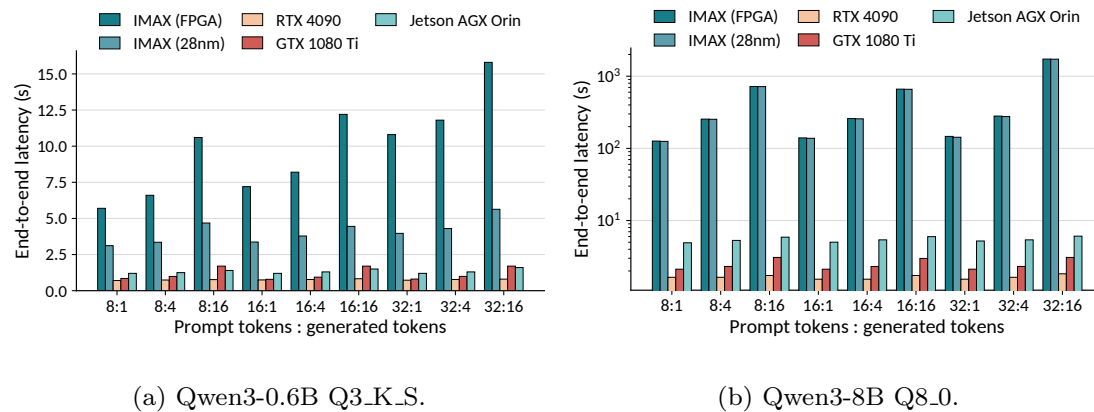
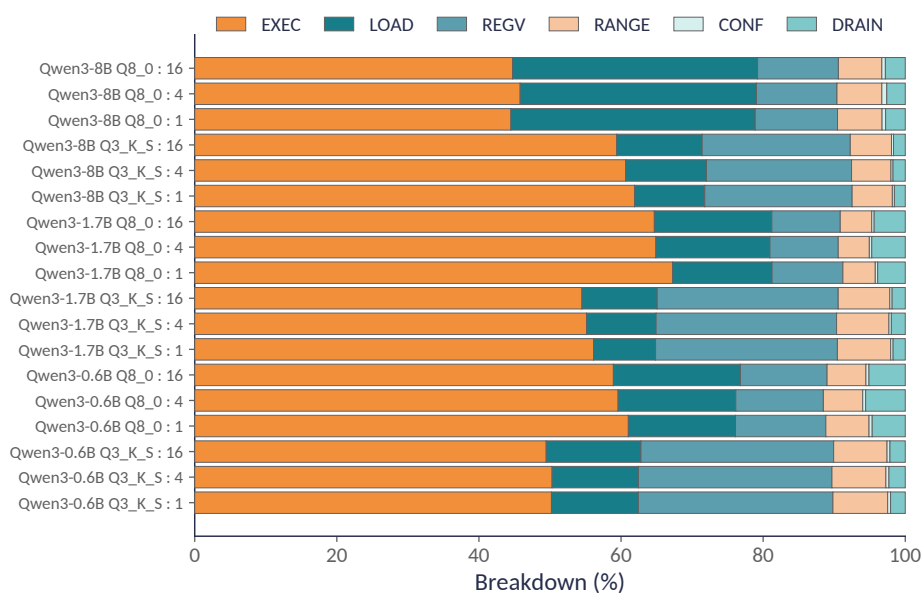


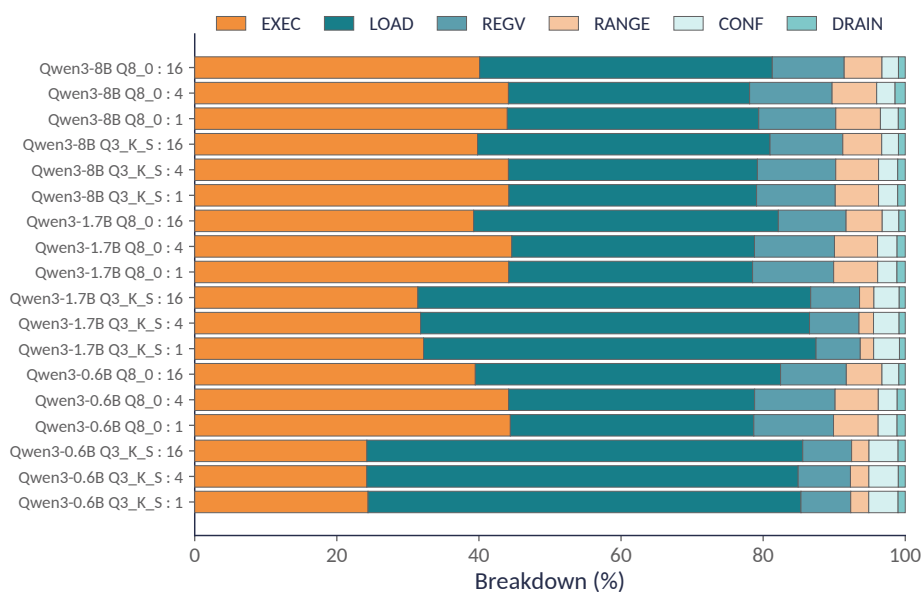
図 4.3 小規模な Qwen3-0.6B Q3_K_S と大規模な Qwen3-8B Q8_0 のエンドツーエンドレイテンシ。横軸の各ラベルは prompt token 数と generated token 数の組を表す。モデル規模と重み転送量の増加に伴う IMAX, GPU, エッジ向けプラットフォーム間の差を示す。

4.3.1 プロンプト処理とデコード

本節のフェーズ境界に関する記述は、図 4.4 に示す Qwen3 測定だけを対象とする。プロンプト処理は大きな演算を含む一方、デコードは小さな演算を繰り返すため固定オーバーヘッドの影響が強い。報告したデコード測定では、LOAD がアクセラレータフェーズの最大部分を占める。これらの測定では、生成される各トークンがパラメータの再転送を必要とする。そのため、演算カーネルがオフロード済みでも転送が支配的になる。このフェーズ差は、プロンプト処理とデコードで異なる最適化対象を必要とする。EXEC の短縮はプロンプト処理を短縮できるが、反復するデコードで現れる転送とホスト側のコストは除去しない。デコードのエンドツーエンド時間を短縮するには、チャンク形成と転送スケジューリングにより転送量または転送・設定回数を減らす必要がある。図 4.4 は両フェーズの実行時間内訳を示す。



(a) プロンプト処理フェーズ.



(b) デコードフェーズ.

図 4.4 フェーズ別実行時間内訳。プロンプト処理では EXEC の割合が大きい一方、デコードではトークンごとの入力転送により LOAD の割合が増える。

このプロンプト処理とデコードの非対称性は、第 5 章で扱うソフトウェアスタック改善の直接の動機である。プロンプト処理が EXEC 支配的でデコードが LOAD 支配的であれば、単一の静的実行方針は両フェーズに適さない。動的に生成長が変わる場合も、チャンク形成とパディングまたは残余フォールバックで処理できる。その効率は、チャンク充填率と転送回数に依存する。

4.3.2 モデルサイズのスケールリング

Qwen3 評価は、0.6B、1.7B、8B の三つのモデルサイズを含む。図 4.5 は、中規模の 1.7B モデルの Q8_0 におけるエンドツーエンドレイテンシと PDP を示す。先に示した 0.6B と 8B の結果と合わせて、三点のスケールリング観点を与える。

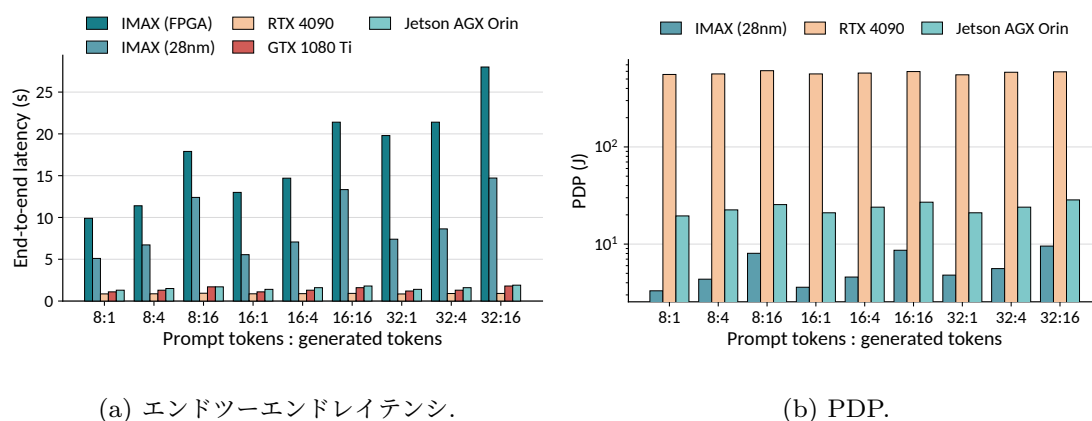


図 4.5 Qwen3-1.7B Q8_0 のエンドツーエンドレイテンシと再計算した PDP。PDP パネルは、上記の FPGA から ASIC への投影および電力推定方法を用いて計算する。

評価した Qwen3 のモデルサイズ掃引では、モデルサイズが 0.6B から 8B へ増えると、パラメータ数と重み転送量が大きく増える。プロンプト処理フェーズでは、行列次元の増加により EXEC 割合が増え、オフロードの利点が出やすい。デコードフェーズでは、各トークンがより多くの重みデータ移動を必要とするため、トークンごとの転送コストが増える。このため、評価したモデルサイズの範囲では

LOAD 割合が増え、フェーズ内訳分析で確認したデコードの転送支配がさらに強まる。図 4.6 は、代表的な Qwen3-0.6B Q3_K_S 条件の PDP と EDP を示す。

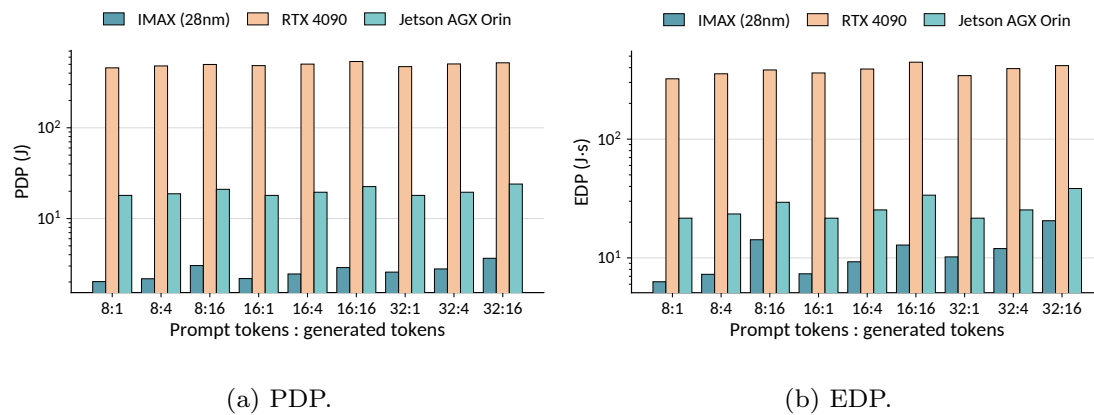
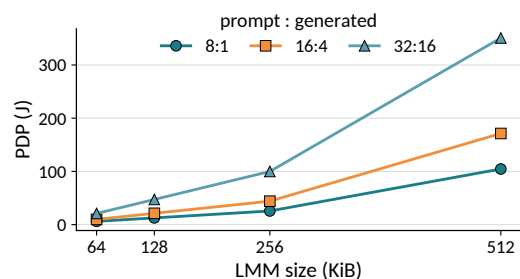


図 4.6 Qwen3-0.6B Q3_K_S の再計算した PDP と EDP。上記の FPGA から ASIC への投影および電力推定方法を用いて計算する。

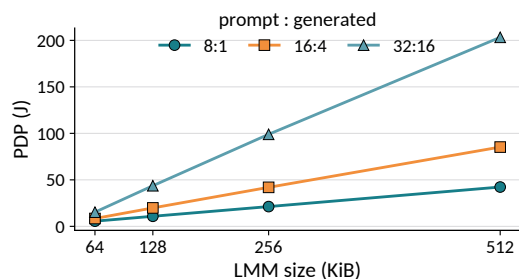
現行の証拠が支持する結論は、エンドツーエンドコストの残りが転送と制御挙動にあるというレイテンシおよびフェーズの観測結果である。図 4.6 の 8:1 条件では、提示した電力モデルの下で IMAX は RTX 4090 に対し PDP を 226 倍、EDP を 51.2 倍改善する。

4.3.3 LMM 容量の設計空間

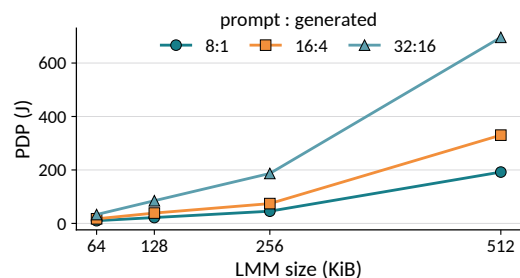
LMM サイズは、オフロード率、転送回数、メモリ電力のトレードオフを決めるハードウェアパラメータである。図 4.7 は、三つの Qwen3 モデルサイズと二つの量子化形式に対する LMM サイズと PDP の関係を示す。評価した 18 曲線では、64 KiB が最小 PDP を与える。ワークロードが収まる容量を超えて LMM を増やすと、静的メモリ電力が増加する。



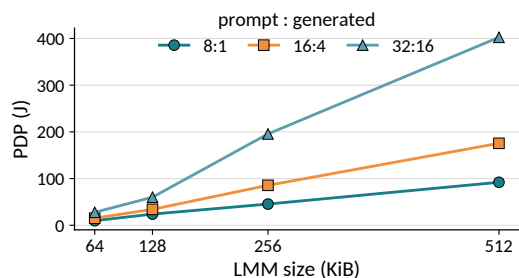
(a) Qwen3-0.6B Q3_K_S



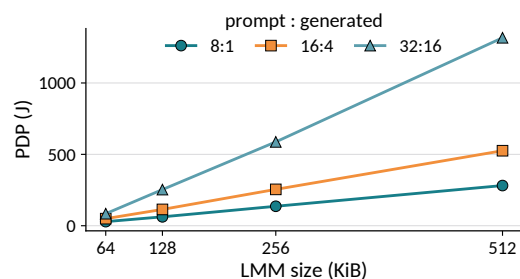
(b) Qwen3-0.6B Q8_0



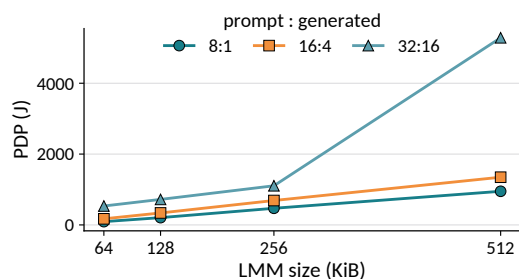
(c) Qwen3-1.7B Q3_K_S



(d) Qwen3-1.7B Q8_0



(e) Qwen3-8B Q3_K_S



(f) Qwen3-8B Q8_0

図 4.7 三つの Qwen3 モデル規模と Q3_K_S / Q8_0 における LMM サイズと PDP の関係。各曲線は、LMM サイズに依存する電力推定の下で一つの prompt token 数と generated token 数の組に対応する。

以上の LLM 結果は，機能上のオフロード成立後に最初に手を入れるべき対象が
チャンク形成と転送挙動であることを示しており，これらは第 5 章で扱う。

第 5 章

CGLA 向けソフトウェアスタック 改善

5.1 スタックレベル改善の必要性

第 4 章の LLM 検討では、カーネルをオフロードしても転送と制御のオーバーヘッドがエンドツーエンド時間に残った。既存 OSS アプリケーションの CPU/GPU 向け実行方針は、IMAX に対して不適切なチャンクと転送挙動を生む場合がある。静的チャンク分割ヒューリスティックは、CGLA のバースト指向実行に対して小さすぎるチャンクを生成し、カーネル起動、DMA レイテンシ、PE の低利用を露出させる場合がある。逆に、大きすぎるチャンクは LMM 容量を超えるかカーネル対応率を下げる。本章では、既存アプリケーション内で実行粒度と転送粒度を選ぶ実行時方針を扱う。IMAX では、各チャンクが LOAD、カーネル設定、EXEC、DRAIN を含む DMA シーケンスを引き起こす。チャンクサイズと転送集約は、ソフトウェアスタック上のパラメータになる。

5.2 Q-Snap

本論文では，CGLA 上の LLM 実行のための量子化を考慮した動的チャンク分割方針 Q-Snap を提案する。評価で示す prompt token 数と generated token 数はモデル系列の単位であり，実行チャンクは一回の CGLA 呼出しで処理する行列の行群である。ここでは，量子化形式と LMM 容量からチャンクサイズを導出する。ベースラインおよび固定チャンクとの比較には，カーネル起動回数とスループットを用いる。基本的な考え方は，行ごとのメモリフットプリント，量子化形式が課すアラインメント制約，局所メモリモジュールの物理容量を同時に考慮してチャンクサイズを決めることである。図 5.1 は従来ベースライン，単純手法，Q-Snap 手法を比較する。

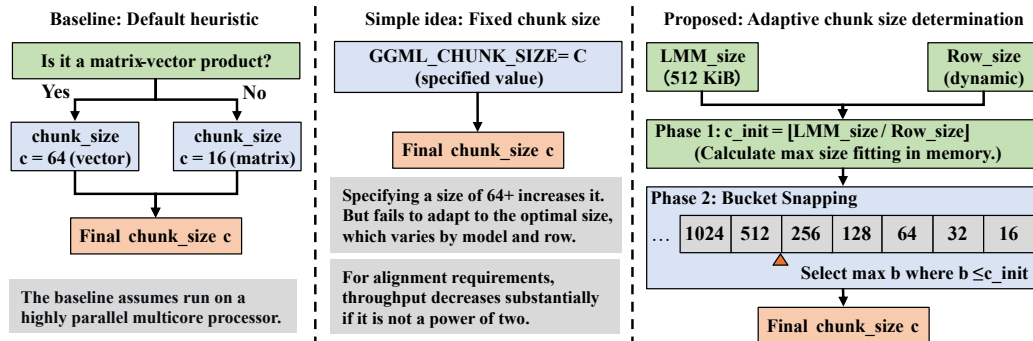


図 5.1 IMAX のドット積カーネルに対する Q-Snap, llama.cpp の既定ヒューリスティック，固定チャンクの比較。

Q-Snap は，CGLA の LMM 容量と転送制約を反映しないアプリケーション既定ヒューリスティックを，行あたりのメモリフットプリント，量子化依存のアラインメント，局所メモリ容量に基づく方針へ置き換える。これらのパラメータは，IMAX 上のバーストサイズ，DMA 回数，局所メモリ適合を決める。Q-Snap は，元の llama.cpp ヒューリスティックではなく，これらの制約からチャンクサイズを

導出する。

5.2.1 チャンクサイズ導出

Q-Snap のチャンクサイズ計算は次のように表せる。 C_{LMM} を利用可能な LMM 容量のバイト数、 K を内積次元とする。量子化形式 q の一ブロックに含まれる要素数を b_q 、対応するブロックサイズのバイト数を s_q とする。量子化された一行のメモリフットプリントは次式で与えられる。

$$f(q, K) = \left\lceil \frac{K}{b_q} \right\rceil s_q. \quad (5.1)$$

一つの LMM に収まる最大行数は次式で与えられる。

$$n_{\max} = \left\lfloor \frac{C_{\text{LMM}}}{f(q, K)} \right\rfloor. \quad (5.2)$$

次に、IMAX のバースト指向の転送粒度に合わせ、チャンクサイズを許容される最大の 2 のべき乗バケットへ切り下げる。バケット集合を $B = \{16, 32, 64, 128, 256, 512, 1024\}$ とすると、最終チャンクサイズは次式で与えられる。

$$n_{\text{chunk}} = \max\{B_i \in B \mid B_i \leq n_{\max}\}. \quad (5.3)$$

式 5.3 は $n_{\max} \geq \min B = 16$ の場合に適用する。 $n_{\max} < 16$ の場合は許容可能な IMAX バケットが存在しないため、その演算はホストフォールバック経路に残す。

式 (5.1)、式 (5.2)、式 (5.3) は、LMM 容量と量子化形式のブロック配置からチャンクサイズを導出する。量子化形式や LMM サイズが変わると、チャンクサイズも自動的に変わる。感度分析では、小さいチャンクが LOAD と DRAIN の反復オーバーヘッドを露出する場合と、大きいチャンクが LMM 制約に違反する場合を比較する。

第 4 章で示した格納コストは、Q-Snap の $f(q, K)$ を決める。Q8_0, Q3_K, Q6_K はそれぞれ 8.50, 3.44, 6.56 bit/weight であり、同じ LMM 容量でも格納できる行数が異なる。各形式のブロックサイズとスケール配置が、式 (5.1) の b_q と s_q を決める。Q-Snap は公称ビット幅ではなく、スケール情報を含む実際の行サイズからチャンクサイズを導出する。 b_q と s_q を定義すれば、他の量子化形式にも同じ計算を適用できる。以下の Q-Snap 定量評価は、Q3_K のみを対象とする。

5.2.2 定量結果

Q-Snap と固定チャンクの比較では、Qwen3-0.6B, Qwen3-1.7B, Qwen3-8B の Q3_K モデルを用いる。各実行は入力 32 tokens, 生成 32 tokens, ホスト 2 threads, IMAX3 の 512 KiB LMM 構成を用いる。ベースラインには llama.cpp の既定の静的チャンクヒューリスティックを用い、各モデルの比較内ではチャンク形成方針だけを変更する。図 5.2 では、Qwen3-0.6B のプロンプト処理スループットが 17.11 tokens/s から 27.70 tokens/s へ向上し、1.62 $\times$ となった。Qwen3-1.7B と Qwen3-8B では、それぞれ 7.5 から 10.2 tokens/s (1.36 $\times$), 1.9 から 2.2 tokens/s (1.16 $\times$) へ向上した。デコードスループットは、三モデルについてそれぞれ 3.9 から 5.6, 1.9 から 2.4, 0.5 から 0.6 tokens/s へ向上した。大規模モデルでは、チャンク形成以外のコストが大きくなるため相対改善率は小さくなる。

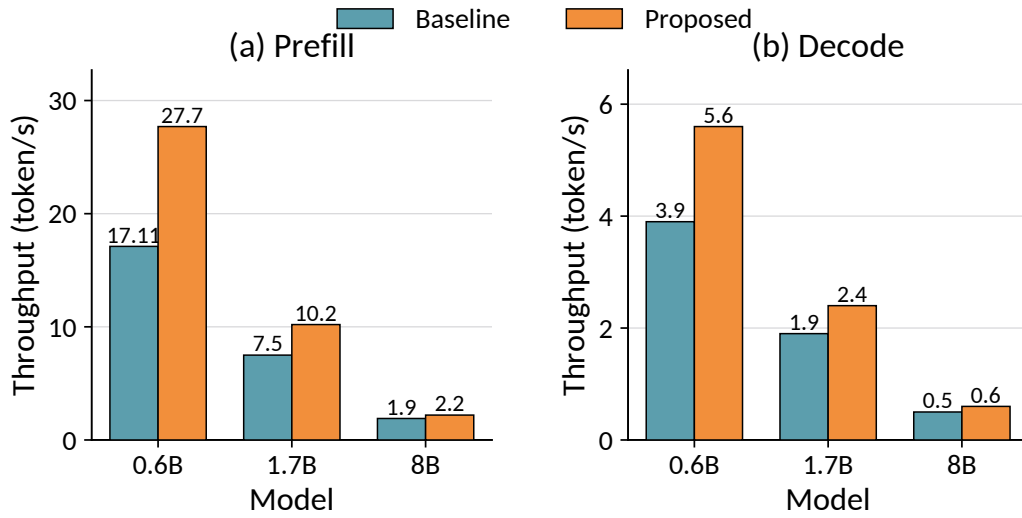


図 5.2 ベースラインチャンク分割方針と Q-Snap のプロンプト処理スループット。量子化形式と LMM 容量に合わせてチャンクを拡大することで、カーネル呼び出しと転送発行を削減した結果を示す。

図 5.3 に固定チャンクサイズの感度を示す。チャンクサイズを 64 から 256 へ増やすと、プロンプト処理スループットは 17.10 から 25.50 tokens/s へ向上し、IMAX カーネル起動回数は 252,602 回から 63,167 回へ減少した。スループットは、チャンクサイズ 512 で 16.80 tokens/s, 4096 で 3.41 tokens/s まで低下した。Q-Snap は、起動回数だけを最小化せず LMM 適合との釣合いを取ることで、33,409 回の IMAX カーネル起動で 27.69 tokens/s を示した。

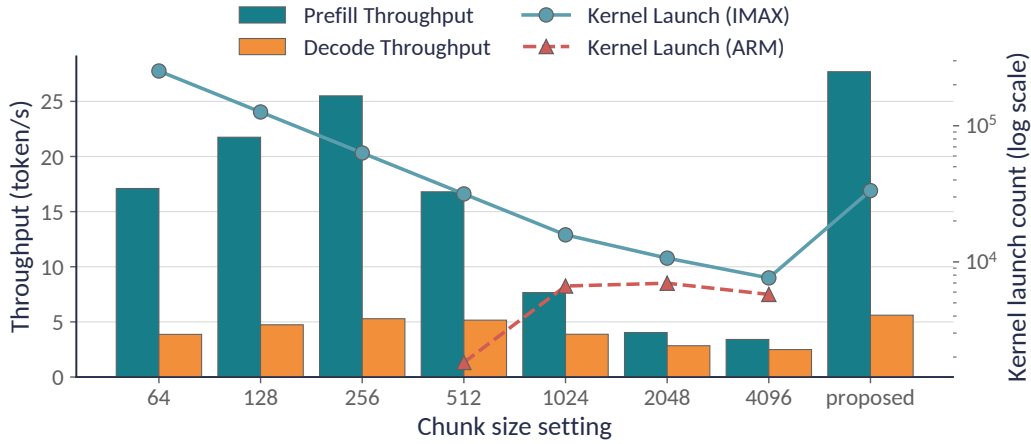


図 5.3 固定チャンクサイズを変化させた場合のスループットとカーネル起動回数。小さすぎるチャンクの起動オーバーヘッドと、大きすぎるチャンクの LMM 不適合との間に動作点があることを示す。

これらの結果は、実行粒度がソフトウェアスタック上の設計パラメータであることを示す。

5.3 プロファイリング基盤

ソフトウェアスタックは、計算時間を転送およびホストオーバーヘッドから分離するためにフェーズ時間を記録する。LOAD, DRAIN, EXEC, 構成オーバーヘッドのフェーズ別時間により、計算律速、転送律速、ホスト律速の挙動を区別する。Q-Snap の選択には、これらのフェーズ測定を用いた。

5.4 スタック改善から得られる設計規則

本章の検討から二つの設計規則が得られる。

第一に、チャンクサイズは、対象アクセラレータの物理制約である量子化形式ごとの行フットプリント、許容バケット集合、利用可能な LMM 容量から導出すべ

きである。これらのパラメータを用いることで、アプリケーション既定のヒューリスティックに対してプロンプト処理スループットは最大 $1.62\times$ 改善した。第二に、改善判断は LOAD, DRAIN, EXEC, ホスト CPU 側オーバヘッドのフェーズ別プロファイリングに基づくべきである。これらのソフトウェア側規則は、既存アプリケーションスタックが実行粒度とプロファイリング境界をどのように選ぶべきかを示す。完全なカーネルマッピング自動化と、より広い演算グラフへの適用は本論文の実装範囲外である。次章では、この方法を ASR と画像生成アプリケーション経路に適用する。

第 6 章

IMAX 上の音声認識と画像生成

本章では、Whisper ASR と Stable Diffusion を IMAX 上の非 LLM アプリケーション経路として扱う。両者は、前章までで述べたホスト・アプリケーション境界を共有するが、支配的なデータ型とカーネル対応範囲条件が異なる。本章では、既存 OSS を保ったローカル実行経路で CGLA の有効範囲を調べる。Whisper は FP16 中心の ASR 条件である。Stable Diffusion は、Q3_K, Q8_0, FP16 のドット積カーネルをオフロードする画像生成条件である。

6.1 GGML 系アプリケーションへの展開

llama.cpp, whisper.cpp, stable-diffusion.cpp は、GGML 系のテンソル表現、行列積呼び出し、量子化データ形式を共有する。IMAX バックエンドは、共通の mul_mat 経路に接続する。GGML 形式が一致する場合は、Q8_0, Q3_K, Q6_K カーネルを再利用する。Whisper と Stable Diffusion 向けには、FP16 カーネルを追加する。アプリケーション固有のエンコーダ、デコーダ、ノイズ除去処理はホスト側に保持し、支配的な行列演算だけをこの境界から段階的にオフロードする。本章では、この共通境界を音声認識と画像生成へ再利用できるかを評価する。

6.2 Whisper ASR

最初の非 LLM アプリケーション経路は Whisper ASR による音声認識である。本節では、データ型と実行時構造が変化した場合にも、ソフトウェアスタック原則を適用できるかを調べる。実装では、インライン FP16-to-FP32 変換、64 bit データ経路上の SIMD 利用、混在実行、パディング除去を持つ FP16 ドット積カーネルを導入した。ホスト CPU 上のプロファイルでは、ドット積演算が FP16 実行時間の 90.6%、Q8_0 実行時間の 87.1% を占めた。この支配的カーネルを IMAX へオフロードする。これにより、Whisper は IMAX バックエンド上の具体的な ASR 実行条件になる。本論文では、Whisper-tiny.en の評価を主な音声認識結果とする。base と small は、より大きなモデルへの拡張として扱う。

6.2.1 FP16 カーネル設計と実装技法

Whisper ワークロードは、支配的テンソル形式が低ビット量子化形式ではなく FP16 である点で LLM 推論と異なる。既存の Q8_0 と Q3_K カーネルはこれらの演算を処理できない。Whisper 用に FP16 ドット積カーネルを設計した。

FP16 カーネルでは複数の実装技法を導入した。第一に、LMM から読み出した FP16 入力を ALU2 のビット操作で FP32 へインライン変換し、専用の型変換回路やホスト側の事前変換を不要にした。第二に、ALU1 と 64 bit データ経路を用いて二つの 32 bit FMA を並列に実行し、FP32 へ変換した値を SIMD で積和する。第三に、列方向マルチスレッド化を用いて、一つの物理 FPU に四つの論理 FMA を時分割で割り当て、FPU レイテンシをパイプライン内で隠す。入力 LMM からストリームとして供給し、変換、積和、部分和削減を PE 列内で連続して行う。この流れを図 6.1 に示す。この FP16 カーネルはレーン当たり 22 PEs を使用し、

IMAX の一次元データフロー上で型変換から最終加算までを実行する。

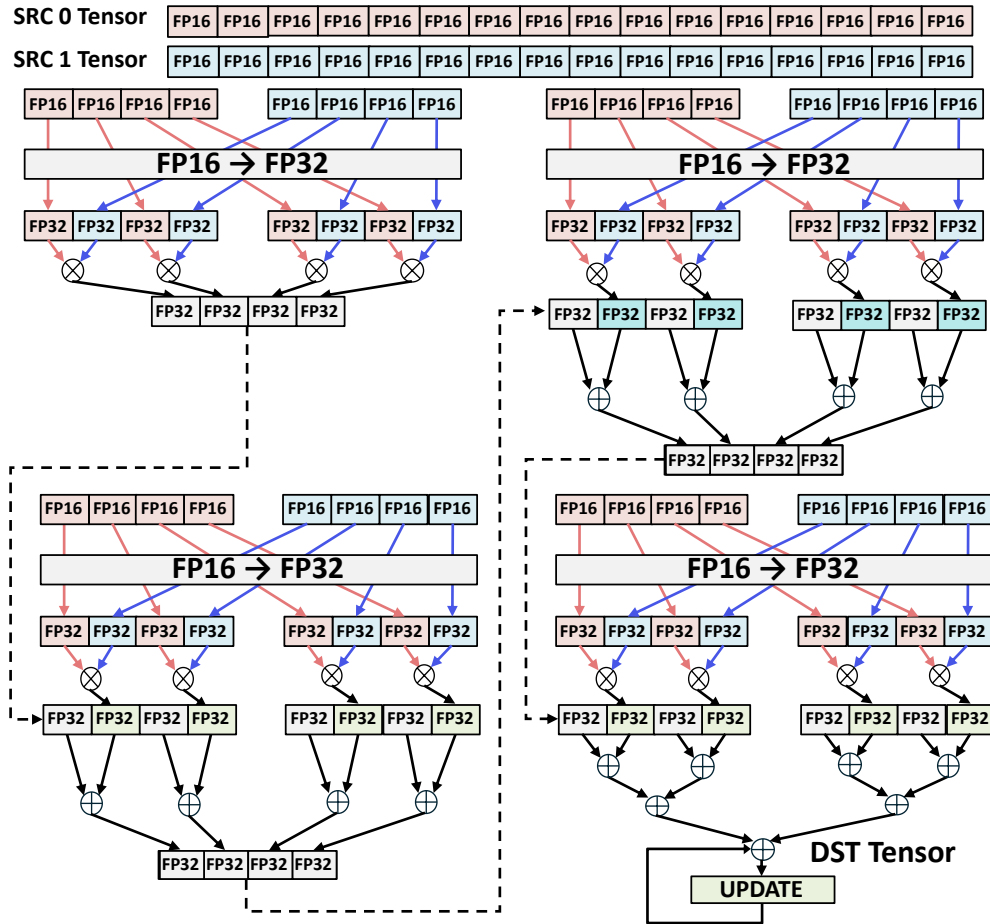


図 6.1 IMAX 上の FP16 ドット積カーネルのデータフロー。ソーステンソルの FP16 入力対を FP32 へ変換して乗算する。PE 列に沿って累積した部分和が出力テンソルを更新する。

第四に、IMAX バースト長の倍数に揃う領域は IMAX で処理し、残余要素はホスト CPU で処理する混在実行方針を採用した。Whisper-tiny.en では主要な行列次元がバースト長の倍数であり、これらのカーネルでは CPU 側残余がゼロになる。第五に、whisper.cpp の CPU SIMD 向け 32 byte アラインメントパディングを除去し、LMM へ転送するパディング量を削減した。

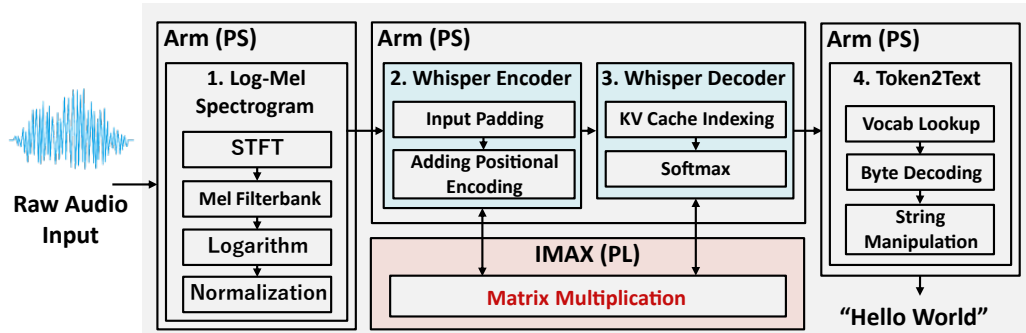


図 6.2 Whisper ASR 処理フロー。入力音声を 80 チャンネルの log-mel spectrogram に変換する。エンコーダは、畳み込みフロントエンドと Transformer 層を通して context embedding を生成する。デコーダはエンコーダ出力と既に生成した token を参照しながら、出力 token を自己回帰的に生成する。

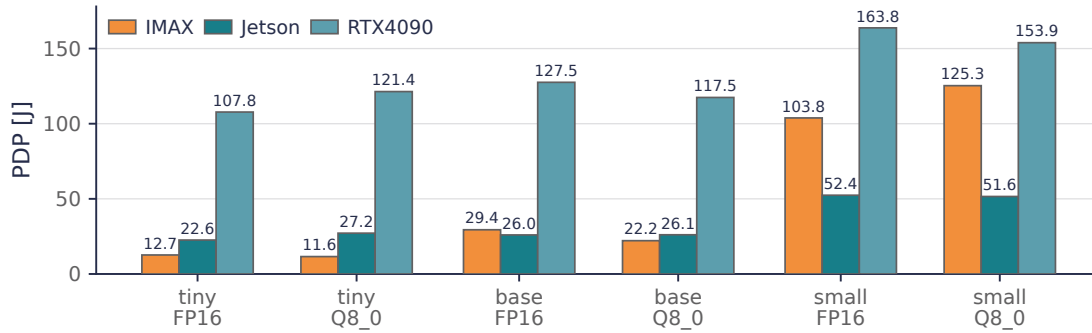
図 6.2 は、Whisper ASR 処理フローを示す。エンコーダとデコーダは、共通バックエンドを通じて FP16 ドット積をオフロードする。ASR 固有の制御はホスト側に残る。この FP16 経路は、FP16 演算がドット積実行時間の 59.0% を占める Stable Diffusion にも必要となる。

6.2.2 エネルギー効率とプラットフォーム比較

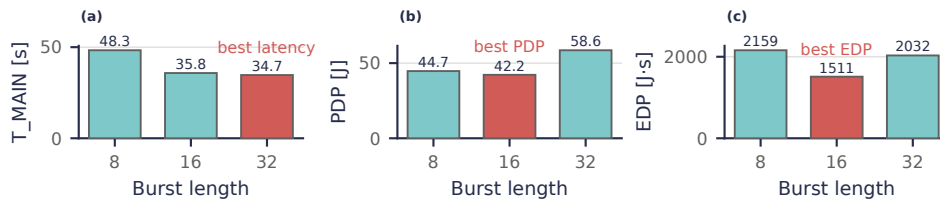
本節の PDP は、第 4 章と同じ FPGA から ASIC への投影および正規化プラットフォーム電力推定方法を用いて計算する。IMAX active の FPGA 実測時間は、28 nm の Design Compiler 推定 [23] に基づき、145 MHz から 840 MHz の条件へ換算する。ホスト CPU 側実行時間は換算しない。本論文の主な ASR 評価では tiny, base, small の三つのモデルサイズと FP16 / Q8_0 量子化を扱う。精度評価には、Whisper-tiny.en と LibriSpeech test-clean の 21 発話を用いた [26]。IMAX FP16 経路の平均正規化 CPU-IMAX 単語レベル文字起こし差は 0.00% であり、CPU と IMAX はともに WER 2.96%, CER 0.74% を記録した。IMAX Q8_0 経路の平均正規化 CPU-IMAX 単語レベル文字起こし差は 0.13% であり、CPU と

IMAX はともに WER 5.03%, CER 1.82% を記録した。

図 6.3 は Whisper の内積カーネルオフロードにおけるクロスプラットフォーム PDP 比較とバースト長トレードオフを示す。Jetson AGX Orin と比較すると、投影後の IMAX は tiny の両形式および base Q8_0 で PDP が低い一方、base FP16 および small の両形式で PDP が高い。RTX 4090 と比較すると、投影後の IMAX は図に示す六つの Whisper 構成のすべてで PDP が低い。IMAX フェーズ測定では、EXEC 部分は FP16 で総実行時間の 60.89%, Q8_0 で 74.70% を占めた。パディング除去、密な DMA パッキング、LMM のダブルバッファを適用した後の測定では、EXEC が最大のフェーズである。バースト長掃引では、バースト長 32 がレイテンシを最小にする一方、バースト長 16 が PDP と EDP を最小にする。



(a) Whisper モデルサイズ別のプラットフォーム間 PDP.



(b) Whisper-tiny.en FP16 のバースト長トレードオフ.

図 6.3 Whisper ASR のプラットフォーム間 PDP 比較とバースト長トレードオフ。PDP 比較は tiny, base, small モデルに対する FPGA から ASIC への投影および正規化プラットフォーム電力推定を用いる。バースト長掃引は、32 KiB LMM, 2 レーン, 2 ホストスレッドの Whisper-tiny.en FP16 条件である。

Whisper 評価では 16 KiB から 256 KiB までの LMM サイズを掃引し、評価したワークロードの PDP は 32 KiB で最小となった。小さい LMM は対応範囲を下げて転送往復を増やす一方、大きい LMM は対応範囲の利点を超えて静的電力を増やす。

表 6.1 は、三つの Whisper モデルサイズについて LMM サイズごとの累積カーネル対応率を示す。32 KiB では、tiny の対応率は 93.8% であるのに対し、base と small は約 66% にとどまる。base と small は、64 KiB で初めて 94% を超える。カーネル数で測った対応率は、わずかに残るホストフォールバックカーネルが実行時間を支配し得るため、そのままレイテンシには対応しない。small Q8_0 では、

64 KiB で対応率が 94% を超える一方、投影レイテンシは 32–64 KiB で約 117s にとどまる。128 KiB でフォールバック時間が減ると、46.84s に低下する。

表 6.1 各 LMM サイズに収まる Whisper カーネルの累積割合。tiny, base, small のモデル規模拡大に伴い、より大きな LMM が必要になる関係を示す。

Model	16 KiB	32 KiB	64 KiB	128 KiB	256 KiB
tiny (78 MB)	66.4	93.8	93.8	100.0	100.0
base (148 MB)	66.5	66.5	94.2	97.1	99.9
small (488 MB)	66.5	66.5	94.4	96.9	99.9

値はカーネル数で測った累積対応率 (%) である。

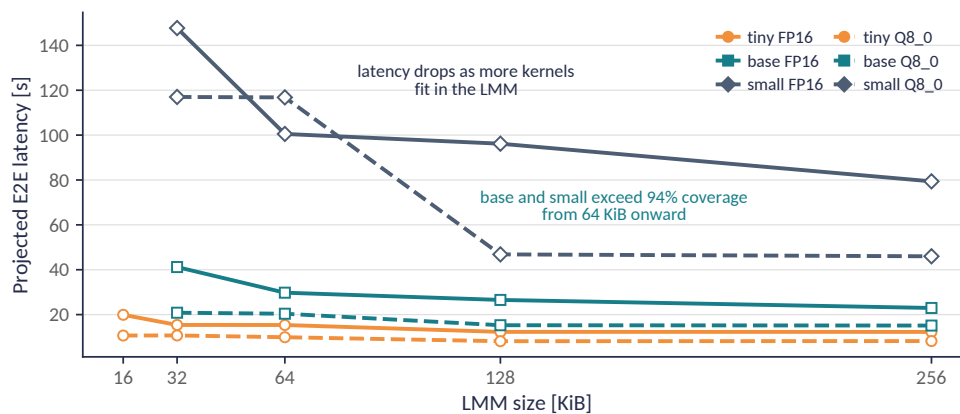


図 6.4 Whisper ASR における LMM 容量に対する射影エンドツーエンドレイテンシ。LMM 容量の増大は base および small モデルのカーネル対応率を高めレイテンシを低減するが、PDP の最適点は LMM の静的電力にも依存する。

FP16 のドット積割合 90.6% から求めた Amdahl 則上限は $10.6\times$ である。Q8_0 のドット積割合 87.1% から求めた上限は $7.8\times$ である。いずれも、CPU プロファイル上でドット積だけを理想的に高速化した場合の上限である。32 KiB LMM を用いる Whisper-tiny.en FP16 では、バースト長 16 が PDP と EDP を最小にする一方、バースト長 32 は高い電力と引き換えにレイテンシを最小にする。したがっ

て、支配的データ型が低ビット量子化から FP16 へ変わる場合、同じソフトウェアスタック境界を保ちつつ、バックエンド拡張と残余要素処理が必要である。

6.3 Stable Diffusion

二つ目の非 LLM アプリケーション経路は Stable Diffusion による画像生成である。本論文では、IMAX バックエンドを `stable-diffusion.cpp` [5] に統合する Stable Diffusion 実行経路を実装し、評価する。この実装では、LLM と Whisper で用いた GGML バックエンド境界を再利用し、`stable-diffusion.cpp` の `mul_mat` 呼び出しから量子化および FP16 カーネルを選択する。この実装では、Q3_K, Q8_0, FP16 のドット積カーネルをオフロードし、残りのノイズ除去パイプラインはホストに残す。

6.3.1 実験設定と定量結果

Stable Diffusion 評価は、145 MHz、各レーン 512 KiB LMM の単一レーン IMAX3 構成を用いた。対象モデルは高速推論用の蒸留版である SD-Turbo である。報告するプラットフォーム比較では Q3_K と Q8_0 構成を用いる。実行経路は同じ IMAX バックエンドから FP16 ドット積演算もディスパッチする。ASIC 投影では、IMAX active 時間を FPGA 動作から、28 nm の Design Compiler 推定 [23] に基づく 840 MHz 条件へ換算する。

RTX 4090 では、SD-Turbo を 512×512 , 1 step で実行し、Q3_K と Q8_0 を各 5 回実行した。実行時間の中央値は Q3_K が 2.45 s, Q8_0 が 2.50 s であった。図 6.5 は、IMAX FPGA, ASIC 投影, Xeon, RTX 4090 の比較を示す。RTX 4090 の PDP には、プラットフォームの公称電力推定を用いる。

図 6.5 における IMAX ASIC 投影の PDP と RTX 4090 の順序は、量子化形式

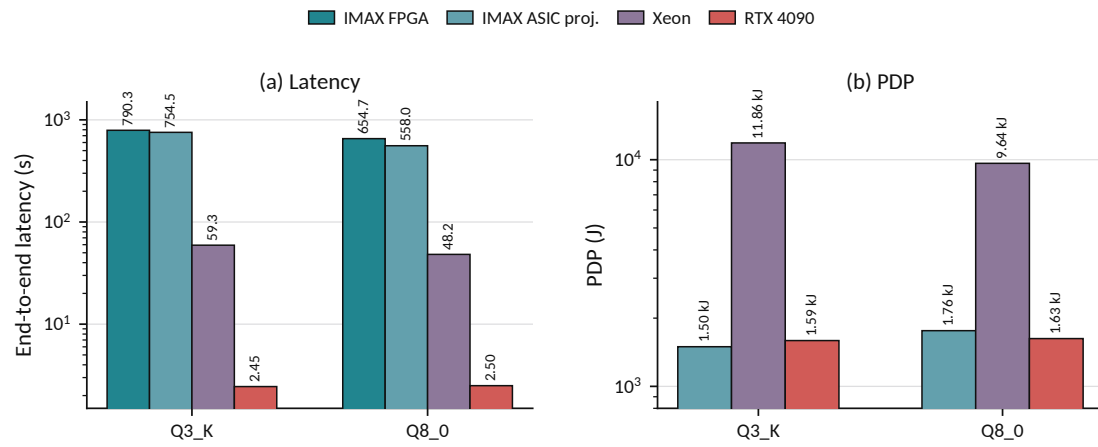


図 6.5 SD-Turbo の IMAX FPGA, 28 nm ASIC 投影, Xeon, RTX 4090 におけるエンドツーエンド時間と PDP。PDP には, 対応する ASIC 投影およびプラットフォームの公称電力推定を用いる。

に依存する。

ワークロード構成は、Stable Diffusion で量子化カーネルと並んで FP16 カーネルを用いる理由を示す。Q3_K, FP16, FP32 は、Stable Diffusion のドット積実行時間の 10.3%, 59.0%, 30.7% をそれぞれ占める。Whisper 向け FP16 カーネルを Stable Diffusion の tensor layout に合わせてコンパイルした。 $K \in \{256, 320, 640, 1280\}$, 出力列数 1 および 16 で CPU 参照値と比較した。これらの試験では CPU 参照値と一致し、guard 領域の破壊はなかった。

画像生成経路は、元の OSS 実行フローを保ったまま、共通の GGML バックエンドとデータ形式別カーネルを第三の OSS アプリケーションへ適用したものである。

6.4 ワークロード横断の考察

LLM, Whisper ASR, Stable Diffusion の比較から、GGML の共通 mul_mat 境界だけでは同じ性能特性にならないことが分かる。LLM では、量子化形式とデコード時の反復転送が主な制約となった。Whisper では、FP16 カーネルと残余要素の CPU 処理が主な制約となった。Stable Diffusion では、FP16 および FP32 の対応範囲が主な制約となった。共通バックエンドの再利用は、実装箇所を限定する。一方、各ワークロードの支配的データ型、テンソル形状、バースト条件に応じたディスパッチとフォールバックが必要である。線形データフローに合う支配的カーネルが存在することが、CGLA の有効条件である。入力と中間データも、LMM に合わせてブロック化できる必要がある。LMM 容量は、各ワークロードでカーネル対応率およびメモリ電力と併せて評価する。

6.5 評価範囲と限界

本章および第 4 章の結果には、四つの前提条件がある。第一に、PDP と EDP は、ワークロード平均電力の実測ではなく、合成に基づく ASIC 推定とプラッ

トフォームの公称電力推定を用いる。第二に，FPGA から ASIC への投影では，IMAX active 時間を Design Compiler に基づく 840 MHz 条件へ換算する。ホスト CPU 側時間は換算しない。第三に，Whisper の精度確認には LibriSpeech test-clean の 21 発話を用いた。レイテンシおよび PDP 比較には，約 10 秒の単一の標準音声クリップを用いた。この分離により統制したプラットフォーム比較を行えるが，入力長の変動性は対象としない。第四に，本論文は，行列演算を中心とする GGML 系アプリケーションを対象とする。クエリ依存の不規則なメモリアクセスや検索データ構造を主成分とする処理は，評価していない。不規則なメモリアクセスを伴うワークロードは今後の課題である。

第 7 章

結論

本論文では, IMAX プロトタイプ上に LLM 推論, Whisper ASR, Stable Diffusion の CGLA 実行経路を実装し, 評価した。評価した低バッチのアプリケーション経路では, PE 演算よりも, 繰り返し発生する LOAD, ホスト処理, 未対応カーネル, 残余 FP32 演算またはアプリケーション制御が制約となった。

三つのアプリケーションは, 元のモデル読み込み, テンソル管理, コマンドラインインターフェース, アプリケーション制御を保持する。GGML オフロード層は, 演算形状とデータ型に応じて CGLA または CPU 実行を選択する。

llama.cpp の評価経路では, デコードが演算スループットよりも転送とホスト側オーバーヘッドに強く制約された。Qwen3 の PDP と EDP は, 本論文で示した FPGA から ASIC への投影および電力推定方法を用いる。Q-Snap はプロンプト処理スループットを $1.62\times$ 改善した。

Whisper-tiny.en の FP16 と Q8_0 経路は, 21 発話の精度評価で対応する CPU 経路と同じ WER および CER を記録した。Jetson AGX Orin と比較すると, 投影後の IMAX は tiny の両形式および base Q8_0 で PDP が低い一方, base FP16 および small の両形式で PDP が高い。RTX 4090 と比較すると, 投影後の IMAX

は図に示す六つの Whisper 構成のすべてで PDP が低い。SD-Turbo では、IMAX ASIC 投影の PDP と RTX 4090 の順序が量子化形式に依存し、評価した FP16 形状は guard 領域を破壊せず CPU 参照値と一致した。

評価したワークロードでは、アクセラレータ上の効率がエンドツーエンド実行に現れるかどうかを、アプリケーション実装とソフトウェアスタック協調が決める。

今後の課題には、ホスト側制御とデータ転送に律速されるデコードフェーズに対する、より広帯域なホストインターコネクトが含まれる。低ビット量子化形式へのハードウェア対応を追加すれば、ホストフォールバックに頼らずより多くのカーネルをアクセラレータ上でネイティブに実行できる。KV キャッシュの扱い、カーネルマッピングのコンパイラ・ツールチェーン改善、評価対象モデルと入力条件の拡大も今後の課題である。

謝辞

本修士研究を進めるにあたり、ご指導とご支援をいただいた中島康彦教授に深く感謝いたします。本論文を審査し、研究に対するコメントをいただいた林優一教授に感謝いたします。張任遠客員教授、菅亦栄助教、Pham Hoai Luan 助教、Le Vu Trung Duong 助教には、助言と励ましをいただきました。

本研究を進める環境を支えてくださった中島研究室の皆様に感謝いたします。中島研究室の元メンバーである、東京都立大学所属の押尾怜穂助教には、本研究に非常に参考となる議論および講義をしていただきました。IMAX, CGLA システム、AI ワークロードに関する共同研究と議論に参加いただいた江藤優氏、竹内歩氏、宗像良文氏に感謝いたします。大学院での研究生活を支えてくださった研究室の先輩、同期、後輩の皆様に感謝いたします。

この期間を支えてくれた仙谷日菜さんに感謝いたします。最後に、常に理解と支援を与えてくれた家族に心より感謝いたします。

参考文献

- [1] T. Akabe, V. T. D. Le, and Y. Nakashima, “IMAX: A power-efficient multilevel pipelined CGLA and applications,” *IEEE Access*, vol. 13, pp. 31899–31911, 2025.
- [2] A. Podobas, K. Sano, and S. Matsuoka, “A survey on coarse-grained reconfigurable architectures from a performance perspective,” *IEEE Access*, vol. 8, pp. 146719–146743, 2020.
- [3] ggml-org, “llama.cpp: LLM inference in C/C++.” <https://github.com/ggml-org/llama.cpp>, 2026. Accessed: July 23, 2026.
- [4] ggml-org, “whisper.cpp: Port of OpenAI’s Whisper model in C/C++.” <https://github.com/ggml-org/whisper.cpp>, 2026. Accessed: July 23, 2026.
- [5] leejet, “stable-diffusion.cpp: Diffusion model inference in C/C++.” <https://github.com/leejet/stable-diffusion.cpp>, 2026. Accessed: July 23, 2026.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30*, pp. 5998–6008, 2017.
- [7] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal,

- A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems 33*, 2020.
- [8] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [9] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 30318–30332, 2022.
- [10] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.
- [11] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” in *Low-Power Computer Vision*, pp. 291–326, Chapman and Hall/CRC, 2022.
- [12] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, pp. 28492–28518,

PMLR, 2023.

- [13] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10674–10685, 2022.
- [14] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [15] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, “EIE: Efficient inference engine on compressed deep neural network,” in *Proceedings of the 43rd Annual International Symposium on Computer Architecture*, pp. 243–254, 2016.
- [16] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [17] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, “Plasticine: A reconfigurable architecture for parallel patterns,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, pp. 389–402, 2017.
- [18] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-L. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy,

- J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snelham, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, “In-datacenter performance analysis of a tensor processing unit,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, pp. 1–12, 2017.
- [19] M. Tanomoto, S. Takamaeda-Yamazaki, J. Yao, and Y. Nakashima, “A CGRA-based approach for accelerating convolutional neural networks,” in *2015 IEEE 9th International Symposium on Embedded Multicore/Many-core Systems-on-Chip*, pp. 73–80, 2015.
- [20] J. Iwamoto, Y. Kikutani, R. Zhang, and Y. Nakashima, “Daisy-chained systolic array and reconfigurable memory space for narrow memory bandwidth,” *IEICE Transactions on Information and Systems*, vol. E103-D, no. 3, pp. 578–589, 2020.
- [21] H. Uetani and Y. Nakashima, “Implementation and evaluation of LLM on a CGLA,” in *2024 Twelfth International Symposium on Computing and Networking (CANDAR)*, pp. 252–258, 2024.
- [22] Y. Eto and Y. Nakashima, “Implementation and performance analysis of LLaMA on a CGLA,” in *Proceedings of the Fifth International Conference on Intelligent Systems and Networks*, pp. 516–525, Springer Nature Singapore, 2025.
- [23] Synopsys, Inc., “DC Ultra: Concurrent timing, area, power and test opti-

- mization.” <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html>. Accessed: May 25, 2025.
- [24] V. J. Reddi *et al.*, “MLPerf inference benchmark,” in *Proceedings of the 47th Annual International Symposium on Computer Architecture*, pp. 446–459, 2020.
- [25] NVIDIA Corporation, “Introducing the GeForce GTX 1080 Ti, the world’s fastest gaming GPU.” <https://www.nvidia.com/en-us/geforce/news/nvidia-geforce-gtx-1080-ti/>, 2017. Accessed: August 20, 2026.
- [26] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “LibriSpeech: An ASR corpus based on public domain audio books,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5206–5210, 2015.

付録 A

発表リスト

査読付きジャーナル論文

1. Takuto Ando, Yu Eto, Ayumu Takeuchi, and Yasuhiko Nakashima, “Efficient Kernel Mapping and Comprehensive System Evaluation of LLM Acceleration on a CGLA,” *IEEE Access*, vol. 13, pp. 199631–199646, November 2025.
2. Takuto Ando, Yu Eto, Ayumu Takeuchi, and Yasuhiko Nakashima, “Design and Evaluation of Energy-Efficient Whisper Dot-Product Kernel Offloading on a CGLA Architecture,” *Concurrency and Computation: Practice and Experience*, vol. 38, no. 13, article e70826, 2026.

査読付き国際会議論文

1. Takuto Ando, Yu Eto, Ayumu Takeuchi, and Yasuhiko Nakashima, “Energy-Efficient Llama3 Acceleration on a CGLA by Offloading Computational Kernels,” *Proceedings of the 2025 IEEE 38th International System-on-Chip Conference (SOCC) PhD and Master Forum*, September 2025.
2. Yu Eto, Takuto Ando, Ayumu Takeuchi, and Yasuhiko Nakashima, “LLM Performance Bottlenecks on CGLA,” *Proceedings of the 2025 IEEE 38th*

- International System-on-Chip Conference (SOCC) PhD and Master Forum*, September 2025.
3. Ayumu Takeuchi, Yu Eto, Takuto Ando, and Yasuhiko Nakashima, “Energy-Efficient FlashAttention Acceleration on CGLA,” *Proceedings of the 2025 IEEE 38th International System-on-Chip Conference (SOCC) PhD and Master Forum*, September 2025.
 4. Takuto Ando, Yu Eto, and Yasuhiko Nakashima, “A Detailed Analysis of LLM Execution on IMAX3 and Initial Evaluation of IMAX4 Prototype for Server Environment,” *Proc. SASIMI 2025*, pp. 8–13, October 2025.
 5. Yu Eto, Takuto Ando, and Yasuhiko Nakashima, “Performance Evaluation of Flan-T5 in CGLA Based Accelerators,” *Proceedings of the 2025 IEEE 38th International System-on-Chip Conference (SOCC)*, October 2025.
 6. Takuto Ando, Yu Eto, Ayumu Takeuchi, and Yasuhiko Nakashima, “Energy-Efficient Hardware Acceleration of Whisper ASR on a CGLA,” *Proceedings of the 2025 Thirteenth International Symposium on Computing and Networking (CANDAR 2025)*, November 2025.
 7. Takuto Ando, Yu Eto, and Yasuhiko Nakashima, “Implementation and Evaluation of Stable Diffusion on a General-Purpose CGLA Accelerator,” *Proceedings of the 2025 IEEE 18th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*, December 2025.
 8. Takuto Ando, Ayumu Takeuchi, Yu Eto, Yoshifumi Munakata, and Yasuhiko Nakashima, “Q-Snap: Quantization-Aware Dynamic Chunking for LLM Execution on a CGLA,” *Proceedings of the Sixth International Conference on Intelligent Systems and Networks (ICISN 2026)*, March 2026.

国内会議論文（日本語）

1. Takuto Ando, Yoshifumi Munakata, and Yasuhiko Nakashima, “Implementation and Evaluation of Comprehensive Software Stack for Diverse AI Workloads on CGLA (in Japanese),” *Proceedings of the 257th System and Architecture Research Symposium*, June 2026.

2. Yoshifumi Munakata, Takuto Ando, and Yasuhiko Nakashima, “Evaluation of Scalability of LLM Execution on CGLA with Wide-Bandwidth Memory (in Japanese),” *Proceedings of the 257th System and Architecture Research Symposium*, June 2026.

付録 B

受賞

1. IEEE CEDA All Japan Joint Chapter SASIMI Young Researcher Award, SASIMI 2025, 2025 年 10 月.
2. Best Paper Award, CANDAR 2025, 2025 年 11 月.
3. Best Paper Award, ICISN 2026, 2026 年 3 月.